

Contents

Módulo 6 - Práctica: seguridad con Spring Security y JWT	3
Práctica 6.1 - Introducción a Spring Security	4
Paso 1 - Añadir la dependencia de Spring Security	4
Paso 2 - Arrancar y ver el “susto”	4
Paso 3 - Autenticarse con la contraseña generada	5
Paso 4 - Ver los filtros en acción	5
Paso 5 - Crear la clase SecurityConfig	5
Paso 6 - Ver el comportamiento sin usuarios	6
Paso 7 - Permitir todos los endpoints temporalmente	7
Paso 8 - Configurar reglas específicas por endpoint	7
Paso 9 - Ver la diferencia entre 401 y 403	8
Paso 10 - Depurar con logs	8
Paso 11 - Errores comunes del ejercicio	9
Paso 12 - Reto resuelto — Endpoint público sin autenticación	9
Resultado esperado global	10
Resumen técnico del ejercicio	10
Conclusión y enlace al siguiente punto	11
Práctica 6.2 - Autenticación con usuarios en memoria	11
Paso 1 - Repasar el estado actual	11
Paso 2 - Añadir un endpoint que devuelva el perfil del usuario	11
Paso 3 - Definir el PasswordEncoder	12
Paso 4 - Definir el UserDetailsService	13
Paso 5 - Cerrar los endpoints	14
Paso 6 - Arrancar y probar sin autenticación	14
Paso 7 - Autenticarse con el usuario ana	15
Paso 8 - Autenticarse con el usuario admin	15
Paso 9 - Probar la autorización por rol	15
Paso 10 - Probar con credenciales incorrectas	16
Paso 11 - Errores comunes del ejercicio	16
Paso 12 - Reto resuelto — Añadir un tercer usuario con roles combinados	17
Resultado esperado global	18
Resumen técnico del ejercicio	18
Conclusión y enlace al siguiente punto	18
Práctica 6.3 - Usuarios en base de datos	18
Paso 1 - Crear el paquete auth	18
Paso 2 - Crear la entidad Rol	19
Paso 3 - Crear la entidad Usuario	19
Paso 4 - Crear los repositorios	20
Paso 5 - Crear el UserDetailsService personalizado	20
Paso 6 - Eliminar los usuarios en memoria	21
Paso 7 - Crear el inicializador de usuarios	21
Paso 8 - Arrancar y verificar	22
Paso 9 - Probar la autenticación	22
Paso 10 - Crear el endpoint de registro	22
Paso 11 - Crear el AuthService y el AuthController	23
Paso 12 - Configurar el endpoint público y probar	23
Paso 13 - Errores comunes del ejercicio	24
Paso 14 - Reto resuelto — Endpoint para cambiar la contraseña	24
Resultado esperado global	25
Resumen técnico del ejercicio	26
Conclusión y enlace al siguiente punto	26
Práctica 6.4 - Autorización por roles	26

Paso 1 - Repasar el estado actual	26
Paso 2 - Activar la seguridad por método	26
Paso 3 - Crear un UsuarioController de administración	27
Paso 4 - Crear el UsuarioService	27
Paso 5 - Añadir reglas de URL y de método combinadas	28
Paso 6 - Probar la autorización con distintos usuarios	29
Paso 7 - Crear un endpoint para consultar el propio perfil	29
Paso 8 - Crear el UsuarioPrincipal (para el punto 6.7)	30
Paso 9 - Probar la expresión de autorización	30
Paso 10 - Escribir tests de autorización	31
Paso 11 - Errores comunes del ejercicio	32
Paso 12 - Reto resuelto — Endpoint de cambio de rol	32
Resultado esperado global	34
Resumen técnico del ejercicio	34
Conclusión y enlace al siguiente punto	34

Práctica 6.5 - Introducción a JWT **34**

Paso 1 - Entender la estructura con un ejemplo	34
Paso 2 - Crear una clase para generar un JWT manualmente	35
Paso 3 - Decodificar el token generado	35
Paso 4 - Modificar el payload sin actualizar la firma	36
Paso 5 - Verificar el token con Java	36
Paso 6 - Probar con distintos payloads	36
Paso 7 - Reflexionar sobre la seguridad	37
Paso 8 - Comparar con un token real	37
Paso 9 - Errores comunes del ejercicio	37
Paso 10 - Experimentar con la expiración	38
Paso 11 - Verificar el token con una clave incorrecta	38
Paso 12 - Reto resuelto — Decodificar un token sin verificar la firma	38
Resultado esperado global	39
Resumen técnico del ejercicio	39
Conclusión y enlace al siguiente punto	39

Práctica 6.6 - Generación de tokens JWT **39**

Paso 1 - Añadir las dependencias de JJWT	39
Paso 2 - Configurar las propiedades de JWT	40
Paso 3 - Crear la configuración de la clave	40
Paso 4 - Crear el JwtService	41
Paso 5 - Crear los DTOs de login	42
Paso 6 - Ampliar el AuthService con login y refresh	43
Paso 7 - Ampliar el AuthController	45
Paso 8 - Configurar la seguridad para permitir login	46
Paso 9 - Arrancar y probar el login	46
Paso 10 - Decodificar el token	47
Paso 11 - Probar el refresh	47
Paso 12 - Errores comunes del ejercicio	48
Paso 13 - Reto resuelto — Endpoint para logout	48
Resultado esperado global	49
Resumen técnico del ejercicio	49
Conclusión y enlace al siguiente punto	49

Práctica 6.7 - Filtro JWT **49**

Paso 1 - Verificar el JwtService y el UsuarioPrincipal	49
Paso 2 - Crear el JwtAuthenticationFilter	50
Paso 3 - Registrar el filtro en la cadena	51
Paso 4 - Desactivar HTTP Basic	52
Paso 5 - Arrancar y probar	52
Paso 6 - Verificar el UsuarioPrincipal	53

Paso 7 - Probar con rol ADMIN	53
Paso 8 - Probar con un token inválido	54
Paso 9 - Probar con la cabecera mal formada	54
Paso 10 - Depurar el filtro	54
Paso 11 - Errores comunes del ejercicio	54
Paso 12 - Reto resuelto: expresión SpEL con el ID del usuario	55
Resultado esperado global	55
Resumen técnico del ejercicio	56
Conclusión y enlace al siguiente punto	56
Práctica 6.8 - Protección de endpoints con JWT	56
Paso 1 - Crear el JwtAuthenticationEntryPoint	56
Paso 2 - Crear el JwtAccessDeniedHandler	57
Paso 3 - Actualizar el SecurityConfig	58
Paso 4 - Añadir un endpoint de gestor	59
Paso 5 - Añadir el rol GESTOR al inicializador	59
Paso 6 - Probar el flujo completo	59
Paso 7 - Probar sin token	60
Paso 8 - Probar el refresh token	60
Paso 9 - Probar el logout	60
Paso 10 - Escribir tests de integración	61
Paso 11 - Errores comunes del ejercicio	62
Paso 12 - Reto resuelto — Endpoint para consultar el propio usuario	62
Resultado esperado global	63
Resumen técnico del ejercicio	63
Conclusión y enlace al siguiente punto	63
Práctica 6.9 - Testing de seguridad	63
Paso 1 - Añadir la dependencia spring-security-test	63
Paso 2 - Crear el test del UsuarioController	64
Paso 3 - Ejecutar los tests	65
Paso 4 - Testear el endpoint de gestor	65
Paso 5 - Testear el endpoint público	66
Paso 6 - Testear el login	67
Paso 7 - Testear el refresh	67
Paso 8 - Testear el logout	68
Paso 9 - Testear con post-processors	68
Paso 10 - Testear con @WithUserDetails	69
Paso 11 - Testear el filtro JWT	70
Paso 12 - Errores comunes del ejercicio	71
Paso 13 - Reto resuelto — Test completo de autorización	71
Resultado esperado global	72
Resumen técnico del ejercicio	72

Módulo 6 - Práctica: seguridad con Spring Security y JWT

Regla de ejecución: realiza los puntos en orden. La práctica 6.1 parte del proyecto final de M5; cada práctica posterior parte del proyecto resultante del punto anterior. Los proyectos de referencia se conservarán en M6/6.1/proyecto hasta M6/6.9/proyecto.

Comandos: los ejemplos usan ./mvnw y sintaxis de shell cuando resulta más legible. En Windows, utiliza mvnw.cmd y adapta las variables de shell al terminal que estés usando.

Práctica 6.1 - Introducción a Spring Security

Contexto del ejercicio: vamos a partir del proyecto final de M5, añadir Spring Security, observar qué hace automáticamente y sustituir después esa configuración implícita por un SecurityFilterChain explícito. El objetivo es terminar el punto con una política sencilla: unas pocas rutas públicas y el resto protegido mediante HTTP Basic, que todavía será un mecanismo temporal de aprendizaje.

Requisitos previos: disponer del proyecto final de M5, Java 17 y el Maven Wrapper del curso. El estado final de esta práctica se conserva en M6/6.1/proyecto y será el punto de partida de la práctica 6.2.

Paso 1 - Añadir la dependencia de Spring Security

Abre pom.xml y añade la dependencia de Spring Security sin eliminar ninguna dependencia heredada de M5:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Guarda el fichero y recarga Maven. Si trabajas desde terminal puedes comprobar que el proyecto sigue compilando con:

```
./mvnw -B -DskipTests compile
```

En Windows utiliza mvnw.cmd si estás ejecutando el comando desde cmd.exe o PowerShell.

Lo importante en este paso es no reemplazar el POM de M5 por uno mínimo. M6 es acumulativo: Spring Security se **añade** al proyecto que ya tiene Web, Validation, JPA, H2, PostgreSQL, OpenAPI y las herramientas de calidad anteriores.

Pregunta de reflexión: ¿Qué crees que pasará con los endpoints actuales al arrancar la aplicación después de añadir el starter?

Respuesta razonada: Spring Security detectará la dependencia y activará su configuración por defecto. Como todavía no hemos declarado nuestras reglas, las rutas quedarán protegidas y necesitaremos autenticarnos para acceder a ellas.

Paso 2 - Arrancar y ver el “susto”

Arranca la aplicación:

```
./mvnw spring-boot:run
```

En el log aparecerá una contraseña generada para desarrollo, similar a:

```
Using generated security password: 3fa2b9e1-8c5d-4a7e-9f2b-1d4e5c6a7b8d
```

```
This generated password is for development use only.
Your security configuration must be updated before running your application in production.
```

La cadena exacta será distinta en cada arranque. Ahora prueba una ruta que en M5 era pública:

```
curl -i http://localhost:8080/api/v1/alumnos
```

La respuesta ya no será la colección de alumnos. Verás un rechazo de autenticación, normalmente acompañado de la cabecera de HTTP Basic:

```
HTTP/1.1 401
WWW-Authenticate: Basic realm="Realm"
...
```

Eso demuestra que la incorporación del starter ha cambiado el comportamiento runtime antes de que hayamos escrito una sola clase de configuración de seguridad.

Pregunta de reflexión: ¿Por qué Spring Security protege todo por defecto en lugar de dejar todo abierto?

Respuesta razonada: Porque el fallo seguro es denegar el acceso hasta que el desarrollador declare qué debe ser público. Abrir todo por defecto haría demasiado fácil publicar accidentalmente información o operaciones sensibles.

Paso 3 - Autenticarse con la contraseña generada

Copia la contraseña que aparece en **ese mismo arranque** y utiliza el usuario user:

```
curl -i -u user:<PASSWORD_DEL_LOG> \
http://localhost:8080/api/v1/alumnos
```

El parámetro -u hace que curl envíe una cabecera HTTP Basic con el usuario y la contraseña. Si las credenciales son correctas, la petición atraviesa la autenticación y puede alcanzar el endpoint.

También puedes abrir una ruta protegida desde el navegador. Dependiendo del cliente, verás un diálogo de autenticación o una respuesta que solicita credenciales.

No copies la contraseña generada a application.properties: sólo existe para este estado inicial de desarrollo. En el siguiente punto definiremos usuarios bajo nuestro control.

Pregunta de reflexión: ¿Qué diferencia conceptual hay entre HTTP Basic y un formulario de login?

Respuesta razonada: HTTP Basic transporta las credenciales mediante la cabecera Authorization en las peticiones. Un formulario es una interfaz y un flujo web que recoge credenciales y normalmente delega también en la infraestructura de autenticación. Son formas diferentes de obtener las credenciales, no identidades distintas.

Paso 4 - Ver los filtros en acción

Para observar la cadena de seguridad, activa temporalmente el nivel DEBUG en la configuración de desarrollo:

```
logging.level.org.springframework.security=DEBUG
```

Reinicia la aplicación y realiza primero una petición sin credenciales y después otra con el usuario generado. En los logs aparecerán mensajes relacionados con la protección de la ruta, el contexto y las decisiones de acceso, por ejemplo:

```
Securing GET /api/v1/alumnos
Secured GET /api/v1/alumnos
Set SecurityContextHolder to AnonymousAuthenticationToken
...
```

Los mensajes concretos pueden variar entre versiones internas de Spring Security, por lo que no debes escribir lógica que dependa de esas cadenas. Úsalos como herramienta de diagnóstico.

Cuando hayas terminado la observación, elimina la propiedad DEBUG o devuelve el nivel de logging al que utilizaba M5. No queremos que el estado final del punto dependa de logging verboso permanente.

Pregunta de reflexión: ¿Qué información aportan estos logs que no obtenemos únicamente mirando el código de estado HTTP?

Respuesta razonada: El código HTTP muestra el resultado final. Los logs permiten seguir el recorrido que llevó a ese resultado: si la petición fue tratada como anónima, qué parte de la cadena intervino y en qué etapa se produjo el rechazo.

Paso 5 - Crear la clase SecurityConfig

Ahora haremos explícita la política. Crea:

src/main/java/es/mecd/demo/miproyecto/config/SecurityConfig.java

con una configuración moderna basada en SecurityFilterChain:

```
package es.mecd.demo.miproyecto.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

Esta configuración expresa de forma explícita lo que hasta ahora observábamos implícitamente: cualquier petición necesita autenticación y utilizamos HTTP Basic.

No uses WebSecurityConfigurerAdapter. Si encuentras un ejemplo que extiende esa clase, pertenece al modelo antiguo de Spring Security y no compila con el stack del curso.

Ejecuta:

```
./mvnw -B test
```

antes de continuar. El propósito de este paso no es todavía abrir rutas ni crear usuarios propios, sino tomar el control de la cadena de seguridad con la API moderna.

Pregunta de reflexión: ¿Qué diferencia hay entre depender de la configuración de seguridad por defecto y declarar nuestro propio SecurityFilterChain?

Respuesta razonada: Con el comportamiento por defecto aceptamos una política genérica decidida por la auto-configuración. Al declarar SecurityFilterChain hacemos explícitas nuestras reglas y podemos evolucionarlas de forma controlada: rutas públicas, roles, CORS, sesión y filtros personalizados.

Paso 6 - Ver el comportamiento sin usuarios

Todavía no hemos creado un UserDetailsService propio ni usuarios de la aplicación. El objetivo de este paso es comprobar la diferencia entre **configurar la cadena** y **configurar la fuente de identidades**.

Arranca de nuevo y prueba una petición con credenciales que no correspondan al usuario válido del entorno:

```
curl -i -u usuario:password \
  http://localhost:8080/api/v1/alumnos
```

La petición debe ser rechazada. Si Spring Boot mantiene un usuario de desarrollo autoconfigurado en este estado, utiliza la contraseña que corresponda al arranque para contrastar el resultado. Lo

importante no es memorizar ese usuario provisional, sino comprobar que el `SecurityFilterChain` y el sistema que carga usuarios son piezas distintas.

En 6.2 dejaremos de depender de la identidad de desarrollo y definiremos explícitamente los usuarios mediante `InMemoryUserDetailsManager`.

Pregunta de reflexión: ¿Dónde buscará Spring Security la información necesaria para validar un usuario cuando definamos nuestro propio `UserDetailsService`?

Respuesta razonada: El proveedor de autenticación delegará la carga de la identidad en el `UserDetailsService`. El mecanismo que presenta las credenciales y la fuente que carga el usuario quedan desacoplados.

Paso 7 - Permitir todos los endpoints temporalmente

Este paso es deliberadamente temporal. Queremos comprobar que las reglas declarativas son realmente las que gobiernan el acceso.

Modifica la autorización de `SecurityConfig` para permitir cualquier petición y desactiva CSRF durante el experimento de API:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .anyRequest().permitAll()
        )
        .csrf(csrf -> csrf.disable())
        .httpBasic(Customizer.withDefaults());

    return http.build();
}
```

Reinicia y prueba:

```
curl -i http://localhost:8080/api/v1/alumnos
```

La ruta vuelve a ser accesible sin credenciales. Eso demuestra que el bloqueo anterior no provenía del controlador ni del servicio: era una decisión de la configuración de seguridad.

No dejes el proyecto en este estado al terminar el punto. `anyRequest().permitAll()` global sólo existe para este experimento y se cerrará en el paso siguiente.

Pregunta de reflexión: ¿Por qué desactivamos CSRF en esta fase de una API REST y por qué eso no significa que CSRF sea una protección inútil?

Respuesta razonada: CSRF protege especialmente escenarios en los que el navegador adjunta automáticamente credenciales como cookies de sesión. Nuestro recorrido acabará usando una API stateless con Bearer tokens enviados explícitamente. Desactivarlo aquí forma parte de ese diseño; en una aplicación web con sesión y cookies la decisión sería distinta.

Paso 8 - Configurar reglas específicas por endpoint

Sustituye el `permitAll()` global por reglas concretas. El objetivo es que documentación, consola H2 y la zona pública puedan consultarse sin autenticación, mientras que el resto de rutas quede protegido:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/public/**").permitAll()
            .requestMatchers("/h2-console/**").permitAll()
            .requestMatchers(
```

```

        "/swagger-ui/**",
        "/swagger-ui.html",
        "/v3/api-docs/**"
    ).permitAll()
    .anyRequest().authenticated()
)
.csrf(csrf -> csrf.disable())
.httpBasic(Customizer.withDefaults());

return http.build();
}

```

En M5 ya trabajamos CORS. No elimines aquella configuración. Spring Security debe integrarse con el contrato acumulativo del proyecto, no sustituirlo.

Prueba una ruta pública cuando exista y una ruta protegida:

```
curl -i http://localhost:8080/api/v1/alumnos
```

El endpoint de alumnos debe exigir autenticación. El paso 12 creará explícitamente `/api/v1/public/info` para probar el caso público.

Pregunta de reflexión: ¿Por qué importa el orden de las reglas y por qué `anyRequest()` debe quedar al final?

Respuesta razonada: Porque `anyRequest()` representa el caso general. Las excepciones públicas o las reglas de rol deben declararse antes. Dejar la regla general al final hace visible que todo lo que no haya coincidido con una regla específica sigue protegido.

Paso 9 - Ver la diferencia entre 401 y 403

Para observar los dos estados podemos introducir temporalmente una regla que exija un rol en una ruta. Por ejemplo:

```
.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
```

Sin autenticación, una petición a una ruta protegida no tiene identidad válida y el resultado esperado es 401. Cuando dispongamos de usuarios con roles en 6.2 podremos reproducir también el caso de una identidad autenticada sin ADMIN, que debe producir 403.

Si necesitas demostrar ya el comportamiento en 6.1, utiliza una regla temporal sobre una ruta existente y restaura después las reglas del paso 8. El estado final de 6.1 no debe contener una regla artificial creada únicamente para provocar el ejemplo.

Pregunta de reflexión: ¿Qué diferencia hay entre 401 y 403 en este contexto?

Respuesta razonada: 401 significa que no existe una autenticación válida para acceder al recurso. 403 significa que la identidad ya está autenticada, pero no posee el rol o permiso que exige la regla.

Paso 10 - Depurar con logs

Vuelve a activar temporalmente:

```
logging.level.org.springframework.security=DEBUG
```

Realiza peticiones y observa mensajes semejantes a:

```

Securing GET /api/v1/alumnos
Secured GET /api/v1/alumnos
Set SecurityContextHolder to AnonymousAuthenticationToken
...
Pre-authenticated entry point called. Rejecting access

```

No todos los mensajes tienen que coincidir carácter por carácter con el ejemplo. Lo que debes aprender a buscar es:

- qué ruta se está protegiendo;
- si Spring Security considera la petición anónima o autenticada;
- si se llegó a la autorización;
- si la cadena rechazó la petición antes del controlador.

Después de terminar el diagnóstico, restaura el nivel de logging anterior.

Pregunta de reflexión: ¿Qué información adicional aportan los logs DEBUG de Spring Security?

Respuesta razonada: Permiten localizar la etapa concreta en la que se tomó la decisión. Son especialmente útiles cuando un 401 o un 403 podría deberse a varias causas distintas y el código del controlador no ha llegado siquiera a ejecutarse.

Paso 11 - Errores comunes del ejercicio

Antes de cerrar 6.1, revisa los fallos más habituales:

Error	Causa habitual	Solución
401 en todos los endpoints	Spring Security protege por defecto o falta una excepción pública	Declarar únicamente las rutas públicas necesarias con <code>permitAll()</code>
La contraseña generada deja de aparecer	La aplicación ya tiene configuración o usuarios explícitos	No depender de la contraseña autogenerada; en 6.2 definiremos usuarios propios
403 cuando esperabas 401	Hay identidad autenticada, pero no tiene permisos	Revisar roles/authorities y la regla aplicada
La configuración no se aplica	El bean de seguridad no está siendo creado o la clase no se detecta	Verificar <code>@Configuration</code> , <code>SecurityFilterChain</code> y el paquete de escaneo
Los logs no muestran información de seguridad	El nivel DEBUG no está activo	Activarlo temporalmente y restaurarlo después
WebSecurityConfigurerAdapter no compila	El ejemplo pertenece a una versión antigua	Usar <code>SecurityFilterChain</code> como bean
El preflight CORS es rechazado	La política CORS no está integrada con la cadena	Conservar e integrar la configuración CORS heredada de M5
Todo ha quedado abierto	Sobrevive <code>anyRequest().permitAll()</code> del experimento	Restaurar las reglas específicas antes de cerrar el punto

Ejecuta de nuevo los tests heredados para detectar regresiones:

```
./mvnw -B test
```

El objetivo es que la introducción de seguridad cambie deliberadamente el acceso HTTP, pero no destruya las capas y contratos funcionales construidos hasta M5.

Paso 12 - Reto resuelto — Endpoint público sin autenticación

Reto: crear `/api/v1/public/info` para devolver información sencilla de la aplicación sin pedir autenticación, manteniendo protegida la API de alumnos.

Crea un controlador en el paquete correspondiente, por ejemplo:

```
package es.mecd.demo.miproyecto.info;
```

```
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/public")
public class PublicInfoController {

    @GetMapping("/info")
    public Map<String, String> info() {
        return Map.of(
            "aplicacion", "mi-proyecto",
            "version", "1.0.0",
            "estado", "activo"
        );
    }
}
```

Comprueba que SecurityConfig contiene la excepción pública:

```
.requestMatchers("/api/v1/public/**").permitAll()
```

Arranca y prueba el endpoint público:

```
curl -i http://localhost:8080/api/v1/public/info
```

Debe responder 200 sin credenciales. A continuación prueba una ruta protegida:

```
curl -i http://localhost:8080/api/v1/alumnos
```

Debe seguir exigiendo autenticación.

Pregunta de reflexión: ¿Qué tendríamos que cambiar si quisiéramos hacer público únicamente /api/v1/public/info y no cualquier futura ruta bajo /api/v1/public/**?

Respuesta razonada: Sustituiríamos el patrón comodín por el matcher exacto de /api/v1/public/info. Así una nueva ruta añadida al mismo controlador no heredaría accidentalmente acceso anónimo.

Resultado esperado global

Al terminar el punto 6.1, el proyecto debe conservar todo lo construido en M5 y añadir:

- spring-boot-starter-security;
- un SecurityConfig basado en SecurityFilterChain;
- HTTP Basic como mecanismo temporal de esta fase;
- reglas explícitas que diferencian rutas públicas y protegidas;
- CSRF desactivado de acuerdo con el recorrido de API REST del módulo;
- la ruta pública /api/v1/public/info;
- ausencia del permitAll() global que utilizamos temporalmente para experimentar;
- logging de Spring Security restaurado a su estado normal después de las pruebas.

Ese estado se conserva en M6/6.1/proyecto y será el punto de partida exacto de 6.2.

Resumen técnico del ejercicio

Durante esta práctica hemos comprobado que añadir spring-boot-starter-security modifica inmediatamente el acceso a la aplicación. Hemos utilizado la credencial provisional de desarrollo para observar HTTP Basic, activado DEBUG para seguir la cadena de filtros y sustituido la configuración implícita por un SecurityFilterChain explícito.

También hemos experimentado con permitAll() y lo hemos retirado antes de cerrar el punto, dejando únicamente las rutas públicas necesarias. La diferencia 401/403 queda preparada para profundizarla en 6.2, cuando ya tendremos usuarios y roles propios.

Conclusión y enlace al siguiente punto

El punto 6.1 introduce la infraestructura sobre la que se construirá el resto de M6. Spring Security intercepta las peticiones antes de los controladores, autentica identidades y aplica reglas de autorización mediante una cadena de filtros configurable.

El proyecto ya no depende únicamente de la configuración segura por defecto: tiene una política declarada con `SecurityFilterChain`. Sin embargo, seguimos utilizando una identidad provisional o de desarrollo. En **6.2 - Autenticación con usuarios en memoria** definiremos explícitamente usuarios, roles y contraseñas mediante `InMemoryUserDetailsManager` y `PasswordEncoder`, y podremos probar de forma controlada tanto 401 como 403.

Práctica 6.2 - Autenticación con usuarios en memoria

Contexto del ejercicio: Vamos a definir usuarios en memoria, cifrar sus contraseñas, asignar roles y comprobar el flujo de autenticación y autorización con HTTP Basic antes de pasar a una fuente persistente de identidades.

Requisitos previos: Tener completado el punto 6.1, con Spring Security y un `SecurityFilterChain` explícito.

Paso 1 - Repasar el estado actual

Antes de añadir usuarios propios, verificamos el cierre de 6.1:

- `spring-boot-starter-security` está presente;
- existe un `SecurityFilterChain`;
- `/api/v1/public/**` es público;
- el resto requiere autenticación;
- el `permitAll()` global experimental ya no existe;
- CORS M5 continúa configurado.

El punto 6.2 parte exactamente del estado conseguido en 6.1 y lo hace evolucionar sin sustituir el proyecto por otro distinto.

Archivos que intervienen:

- `src/main/java/es/meecd/demo/miproyecto/config/SecurityConfig.java`
- `src/main/java/es/meecd/demo/miproyecto/config/CorsConfig.java`
- `pom.xml`

Ejecuta la suite antes de continuar:

```
./mvnw -B test
```

El proyecto mantiene el contrato acumulado; todavía no hay usuarios controlados por el curso.

Pregunta de reflexión: ¿Qué diferencia hay entre definir una política de autorización y definir de dónde salen los usuarios?

Respuesta razonada: La política decide qué identidades pueden acceder; el `UserDetailsService` decide cómo se obtiene una identidad durante la autenticación. Podemos cambiar de memoria a base de datos sin reescribir el significado de una regla de autorización.

Antes de modificar nada, ejecuta la suite heredada para comprobar que partes de un M6.1 sano:

```
./mvnw -B test
```

Paso 2 - Añadir un endpoint que devuelva el perfil del usuario

Creamos:

```
src/main/java/es/meecd/demo/miproyecto/common/controller/PerfilController.java
```

```
package es.mecd.demo.miproyecto.common.controller;

import java.util.Map;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/perfil")
public class PerfilController {

    @GetMapping
    public Map<String, Object> perfil(Authentication authentication) {
        return Map.of(
            "usuario", authentication.getName(),
            "authorities", authentication.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority)
                .toList());
    }
}
```

Esto demuestra que Spring puede inyectar el Authentication ya resuelto por la cadena. El controlador no vuelve a validar la contraseña.

Después, prueba:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/perfil
```

Una vez completados los pasos siguientes, la respuesta será:

```
{
  "usuario": "ana",
  "authorities": ["ROLE_USER"]
}
```

Error posible. authentication nulo porque la ruta se dejó pública accidentalmente. **Corrección.** Mantener /api/v1/perfil dentro de anyRequest().authenticated().

Paso 3 - Definir el PasswordEncoder

El encoder es un bean estable; no pertenece sólo al estado temporal de usuarios en memoria porque 6.3 también lo necesitará.

En SecurityConfig:

```
@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}
```

No guardamos ana123 directamente. La contraseña que conoce el usuario es una entrada; el sistema conserva y compara una representación codificada.

Puedes comprobarlo con:

```
String hash1 = passwordEncoder.encode("ana123");
String hash2 = passwordEncoder.encode("ana123");

assert !hash1.equals(hash2);
assert passwordEncoder.matches("ana123", hash1);
assert passwordEncoder.matches("ana123", hash2);
```

Que dos hashes sean diferentes no significa que una contraseña sea incorrecta: BCrypt incorpora salt.

Error posible. NoSuchBeanDefinitionException: PasswordEncoder. **Corrección.** Comprobar que el método está anotado con @Bean y que su clase está bajo el escaneo de Spring.

Pregunta de reflexión: ¿Por qué matches funciona aunque dos llamadas a encode puedan producir hashes diferentes?

Respuesta razonada: Porque el hash incorpora un salt. matches extrae del hash codificado la información necesaria para verificar la contraseña original; no necesita que una segunda llamada a encode produzca exactamente la misma cadena.

Guarda la configuración y comprueba que el proyecto sigue compilando:

```
./mvnw -B -DskipTests compile
```

Paso 4 - Definir el UserDetailsService

Creamos una configuración explícitamente temporal:

src/main/java/es/mecd/demo/miproyecto/config/InMemoryUserConfig.java

@Configuration

```
public class InMemoryUserConfig {

    @Bean
    UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
        UserDetails ana = User.builder()
            .username("ana")
            .password(passwordEncoder.encode("ana123"))
            .roles("USER")
            .build();

        UserDetails admin = User.builder()
            .username("admin")
            .password(passwordEncoder.encode("admin123"))
            .roles("ADMIN", "USER")
            .build();

        return new InMemoryUserDetailsManager(ana, admin);
    }
}
```

Transición a usuarios controlados. A partir de aquí dejamos de depender del usuario aleatorio de desarrollo generado por Boot y controlamos las identidades didácticas.

ana tiene ROLE_USER; admin tiene ROLE_ADMIN y ROLE_USER.

Error posible. Escribir .roles("ROLE_ADMIN"). **Corrección.** Con roles(...) se usa "ADMIN"; Spring añade ROLE_.

Pregunta de reflexión: ¿Y el usuario ana necesita el rol ADMIN para consultar un recurso ordinario?

Respuesta razonada: No, salvo que esa ruta se haya protegido expresamente con ADMIN. Un usuario autenticado puede acceder a las rutas que sólo exigen autenticación y recibe 403 únicamente en las que requieren un rol que no posee.

Después de registrar InMemoryUserDetailsManager, vuelve a compilar:

```
./mvnw -B -DskipTests compile
```

Paso 5 - Cerrar los endpoints

Evolucionamos el SecurityFilterChain:

http

```
.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/public/**").permitAll()
    .requestMatchers("/h2-console/**").permitAll()
    .requestMatchers(
        "/swagger-ui/**",
        "/swagger-ui.html",
        "/v3/api-docs/**").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/gestor/**")
        .hasAnyRole("GESTOR", "ADMIN")
    .anyRequest().authenticated())
.httpBasic(Customizer.withDefaults());
```

- Conservamos `cors(Customizer.withDefaults())` porque M5 ya tiene CORS funcional.
- `sameOrigin()` permite que la consola H2 pueda mostrarse dentro de frame en el estado de desarrollo.
- No añadimos `permitAll()` global.
- Swagger/OpenAPI sigue accesible para poder inspeccionar el contrato del API durante el curso.

Importante. H2 público es una comodidad del entorno pedagógico. No debe extrapolarse sin más a producción.

Pregunta de reflexión: ¿Qué pasaría si `anyRequest()` apareciera antes que las reglas concretas?

Respuesta razonada: Las reglas posteriores podrían quedar inalcanzables o la configuración sería inválida, porque `anyRequest()` representa el caso general. Las reglas específicas deben declararse antes y cerrar con `anyRequest()` al final.

Ejecuta los tests antes de arrancar manualmente:

```
./mvnw -B test
```

Paso 6 - Arrancar y probar sin autenticación

Una vez materializado el proyecto completo:

```
./mvnw spring-boot:run
```

En otra terminal:

```
curl -i http://localhost:8080/api/v1/alumnos
curl -i http://localhost:8080/api/v1/public/info
```

Resultados esperados.

- `/api/v1/alumnos` → 401 Unauthorized y, en el estado Basic, cabecera `WWW-Authenticate`.
- `/api/v1/public/info` → 200 OK.

Diagnóstico. Si ambos devuelven 200, una regla demasiado amplia ha abierto el API. Si ambos devuelven 401, la excepción pública no se está aplicando.

Pregunta de reflexión: ¿Por qué el mismo SecurityFilterChain puede producir 200 y 401 según la URL?

Respuesta razonada: Porque la cadena evalúa reglas distintas según el matcher de la petición. Una ruta permitAll puede responder 200 sin identidad, mientras otra con authenticated() exige credenciales válidas.

Comprueba en una sola secuencia una ruta protegida y otra pública:

```
curl -i http://localhost:8080/api/v1/alumnos
curl -i http://localhost:8080/api/v1/public/info
```

Paso 7 - Autenticarse con el usuario ana

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/alumnos
curl -i -u ana:ana123 http://localhost:8080/api/v1/perfil
```

La consulta ordinaria debe estar autorizada y el perfil debe mostrar:

```
{
  "usuario": "ana",
  "authorities": ["ROLE_USER"]
}
```

El orden de una colección de authorities no debe usarse como contrato si hay varias; comprobaremos contenido, no una ordenación accidental.

Pregunta de reflexión: ¿Qué authority tiene ana y de dónde proviene el prefijo ROLE_?

Respuesta razonada: Al construirla con .roles("USER"), Spring crea la authority ROLE_USER. El prefijo es una convención de roles de Spring Security; .authorities(...) no lo añade automáticamente.

Compara un recurso ordinario con el perfil usando las mismas credenciales:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/alumnos
curl -i -u ana:ana123 http://localhost:8080/api/v1/perfil
```

Paso 8 - Autenticarse con el usuario admin

```
curl -i -u admin:admin123 http://localhost:8080/api/v1/perfil
```

El perfil contiene ROLE_ADMIN y ROLE_USER.

En este diseño conserva también USER porque En este diseño didáctico ADMIN puede ejercer las capacidades ordinarias de USER además de las administrativas. No es una regla intrínseca de Spring Security; es una decisión de roles del curso.

Pregunta de reflexión: ¿Es obligatorio que todo usuario con rol ADMIN tenga también el rol USER?

Respuesta razonada: No. Los roles no forman una jerarquía automática. Si queremos que ADMIN herede permisos de USER podemos asignar ambos roles o configurar una jerarquía explícita, pero no debe suponerse.

Paso 9 - Probar la autorización por rol

Primero con un usuario autenticado pero no autorizado:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/admin/usuarios
```

La petición debe devolver 403 Forbidden.

Luego:

```
curl -i -u admin:admin123 http://localhost:8080/api/v1/admin/usuarios
```

En este momento el endpoint administrativo aún no existe. El usuario admin supera la autorización y la resolución del recurso puede terminar en 404 Not Found. Lo importante de este paso es que **ya no es 403**.

Diferencia que debes comprobar.

- 401: no existe una autenticación válida.
- 403: existe identidad, pero carece del permiso.
- 404: la seguridad permitió continuar, pero no hay recurso/mapping coincidente.

Pregunta de reflexión: ¿Por qué un 404 después de autenticarse como admin es una señal distinta a un 403?

Respuesta razonada: Un 403 demuestra que la ruta existe pero la autorización la bloquea. Un 404 puede indicar simplemente que no existe el endpoint; por tanto no sirve como evidencia de que la regla de rol esté funcionando.

Prueba los dos lados de la regla administrativa:

```
curl -i -u ana:ana123 http://localhost:8080/api/v1/admin/usuarios
curl -i -u admin:admin123 http://localhost:8080/api/v1/admin/usuarios
```

Paso 10 - Probar con credenciales incorrectas

```
curl -i -u ana:contraseñaIncorrecta \
http://localhost:8080/api/v1/alumnos
```

```
curl -i -u pedro:loQueSea \
http://localhost:8080/api/v1/alumnos
```

Ambos casos deben devolver 401.

No devolvemos “el usuario existe pero la contraseña falla” frente a “el usuario no existe”. Dar respuestas diferentes facilitaría enumerar cuentas válidas.

Pregunta de reflexión: ¿Qué información podría obtener un atacante si la API distinguiera públicamente ambos casos?

Respuesta razonada: Podría inferir qué usuarios existen y cuáles no, facilitando enumeración de cuentas. Por eso el login debe responder de forma uniforme a credenciales inválidas.

Prueba tanto una contraseña incorrecta como un usuario inexistente:

```
curl -i -u ana:incorrecta http://localhost:8080/api/v1/alumnos
curl -i -u nadie:incorrecta http://localhost:8080/api/v1/alumnos
```

Paso 11 - Errores comunes del ejercicio

Error	Causa probable	Corrección
401 siempre	credenciales mal enviadas o usuario inexistente	revisar <code>curl -u</code> y <code>UserDetailsService</code>
403 con usuario autenticado	no posee el rol necesario	revisar <code>.roles(...)</code> y la regla de URL
la contraseña no coincide	no se codificó con el mismo <code>PasswordEncoder</code>	crearla con <code>passwordEncoder.encode(...)</code>
<code>NoSuchBeanDefinitionException</code> : <code>PasswordEncoder</code> ciclo al arrancar	falta el bean	declarar el <code>@Bean</code> estable
la aplicación no compila	dependencia circular entre configuración/beans import equivocado o ausente	separar responsabilidades e inyección usar imports de Spring Security actuales
no aparecen logs de seguridad búsqueda de usuario incorrecta	nivel <code>DEBUG</code> no activado <code>UserDetailsService</code> no resuelve el username esperado	activarlo sólo para diagnóstico revisar implementación/datos

Error	Causa probable	Corrección
CORS preflight falla se usa NoOpPasswordEncoder	Security intercepta OPTIONS sin integrar CORS atajo inseguro	conservar <code>.cors(...)</code> y probar preflight eliminarlo y usar el encoder del curso

Comprobación de cierre. No convertir un error de test en “solución” eliminando el test heredado.

Tras corregir cualquiera de los errores de la tabla, vuelve a ejecutar la suite:

```
./mvnw -B test
```

Paso 12 - Reto resuelto — Añadir un tercer usuario con roles combinados

El reto consiste en un usuario gestor con GESTOR y USER, y una ruta accesible por GESTOR o ADMIN.

1. Evolucionar InMemoryUserConfig.

```
UserDetails gestor = User.builder()
    .username("gestor")
    .password(passwordEncoder.encode("gestor123"))
    .roles("GESTOR", "USER")
    .build();

return new InMemoryUserDetailsManager(ana, admin, gestor);
```

2. Crear el endpoint.

src/main/java/es/meecd/demo/miproyecto/auth/GestorController.java

```
@RestController
@RequestMapping("/api/v1/gestor")
public class GestorController {

    @GetMapping("/documentos")
    public Map<String, String> documentos() {
        return Map.of("mensaje", "Lista de documentos");
    }
}
```

3. Autorizar cualquiera de los dos roles.

```
.requestMatchers("/api/v1/gestor/**")
.hasAnyRole("GESTOR", "ADMIN")
```

4. Probar.

```
# Ana (USER) -> 403
curl -i -u ana:ana123 \
  http://localhost:8080/api/v1/gestor/documentos
```

```
# Gestor -> 200
curl -i -u gestor:gestor123 \
  http://localhost:8080/api/v1/gestor/documentos
```

```
# Admin -> 200
curl -i -u admin:admin123 \
  http://localhost:8080/api/v1/gestor/documentos
```

Pregunta de reflexión: ¿Qué diferencia existe entre `hasRole("ADMIN")` y `hasAnyRole("GESTOR", "ADMIN")`?

Respuesta razonada: La primera exige una única role authority `ROLE_ADMIN`; la segunda acepta cualquiera de las dos, `ROLE_GESTOR` o `ROLE_ADMIN`.

Estos estados tienen un propósito temporal y su evolución está definida en los puntos siguientes:

- los usuarios en memoria se eliminan en 6.3, cuando `UserDetailsService` pasa a cargar identidades de base de datos;
- HTTP Basic continúa durante los puntos intermedios y se retira en 6.7, cuando entra en funcionamiento el filtro JWT;
- `PasswordEncoder` permanece porque también es necesario para registrar y verificar contraseñas persistentes.

Resultado esperado global

- Un bean `PasswordEncoder` y usuarios en memoria `ana`, `admin` y `gestor` con los roles previstos.
- Un `SecurityFilterChain` que deja públicas sólo las rutas expresamente permitidas y protege el resto.
- Un endpoint de perfil que permite observar la identidad autenticada.
- Pruebas manuales de 200, 401 y 403 con HTTP Basic.

Resumen técnico del ejercicio

- `UserDetails`, `UserDetailsService` e `InMemoryUserDetailsManager`.
- `PasswordEncoder` y `BCrypt/DelegatingPasswordEncoder`.
- Roles/authorities y prefijo `ROLE_`.
- HTTP Basic, identidad en el controlador y diferencia 401/403.

Conclusión y enlace al siguiente punto

Los usuarios en memoria permiten aprender el modelo de Spring Security, pero son temporales. En 6.3 los sustituiremos por identidades persistentes sin cambiar los conceptos de autenticación y autorización que acabamos de practicar.

Práctica 6.3 - Usuarios en base de datos

Contexto del ejercicio: Vamos a sustituir los usuarios en memoria por usuarios y roles persistidos en base de datos, implementar un `UserDetailsService` personalizado y añadir registro y cambio de contraseña.

Requisitos previos: Tener completado el punto 6.2 con usuarios en memoria, `PasswordEncoder` y reglas de acceso funcionando.

Paso 1 - Crear el paquete auth

Consolidamos bajo:

```
src/main/java/es/meecd/demo/miproyecto/auth/
```

las entidades, repositorios, servicio de identidad, DTOs y operaciones de autenticación.

El `GestorController` creado en el reto 6.2 ya está en este paquete y se conserva.

La organización por funcionalidad sigue el patrón de M5: los elementos de una misma capacidad permanecen juntos.

El paquete `auth` agrupa las nuevas piezas de identidad y autenticación sin duplicar las capas existentes.

Paso 2 - Crear la entidad Rol

Archivo:

src/main/java/es/mecd/demo/miproyecto/auth/Rol.java

```
@Entity
@Table(name = "roles")
public class Rol {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String nombre;

    @Column(length = 200)
    private String descripcion;

    @ManyToMany(mappedBy = "roles")
    private Set<Usuario> usuarios = new HashSet<>();

    protected Rol() {
    }

    public Rol(String nombre, String descripcion) {
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    // getters/setters
}
```

Rol no define @JoinTable; es el lado inverso.

Error posible. Definir @JoinTable también aquí. **Corrección.** Sólo Usuario. roles es propietario.

Pregunta de reflexión: ¿Qué ocurriría si ambos lados intentasen crear su propia tabla de unión?

Respuesta razonada: Se modelarían dos relaciones distintas o aparecería un esquema incoherente. En una relación bidireccional @ManyToMany, un lado es propietario con @JoinTable y el otro referencia mediante mappedBy.

Paso 3 - Crear la entidad Usuario

Archivo:

src/main/java/es/mecd/demo/miproyecto/auth/Usuario.java

Campos:

- id;
- username único;
- password codificada;
- email único;
- activo;
- fechaCreacion;
- roles.

Relación:

```

@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
    name = "usuarios_rols",
    joinColumns = @JoinColumn(name = "usuario_id"),
    inverseJoinColumns = @JoinColumn(name = "rol_id"))
private Set<Rol> roles = new HashSet<>();

```

Hibernate podrá construir tres tablas: usuarios, roles y usuarios_rols.

Nota técnica. EAGER se mantiene porque este modelo necesita authorities durante autenticación. En un dominio más pesado evaluaríamos LAZY + consulta explícita.

Pregunta de reflexión: ¿Por qué Set encaja mejor que List para roles?

Respuesta razonada: Porque un usuario no debería tener el mismo rol duplicado. Set expresa esa unicidad en el modelo Java y evita duplicados accidentales antes incluso de llegar a la base de datos.

Paso 4 - Crear los repositorios

RolRepository:

```

public interface RolRepository extends JpaRepository<Rol, Long> {
    Optional<Rol> findByNombre(String nombre);
    boolean existsByNombre(String nombre);
}

```

UsuarioRepository:

```

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByUsername(String username);
    boolean existsByUsername(String username);
    boolean existsByEmail(String email);
}

```

Spring Data deriva las consultas a partir de sus nombres y mantiene los detalles SQL fuera de la capa de seguridad.

Podemos localizar una identidad por el mismo username que recibe Spring Security y validar duplicados antes de guardar.

Paso 5 - Crear el UserDetailsService personalizado

Creamos UsuarioDetailsService.

```

@Service
public class UsuarioDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioDetailsService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username) {
        Usuario usuario = usuarioRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Usuario no encontrado: " + username));

        List<GrantedAuthority> authorities = usuario.getRoles().stream()
            .map(rol -> (GrantedAuthority)

```

```

        new SimpleGrantedAuthority("ROLE_" + rol.getNombre()))
        .toList();

    return User.builder()
        .username(usuario.getUsername())
        .password(usuario.getPassword())
        .authorities(authorities)
        .accountExpired(false)
        .accountLocked(!usuario.isActivo())
        .credentialsExpired(false)
        .disabled(!usuario.isActivo())
        .build();
    }
}

```

Un rol persistido como ADMIN se expone a Spring Security como ROLE_ADMIN.

Error posible. Guardar ROLE_ADMIN y volver a prefijarlo. **Corrección.** En nuestra base se guardan nombres sin ROLE_.

Paso 6 - Eliminar los usuarios en memoria

Al terminar este paso, la transición queda cerrada:

Eliminamos físicamente:

config/InMemoryUserConfig.java

No se elimina:

```

@Bean
PasswordEncoder passwordEncoder()

```

porque la autenticación persistente y el registro siguen necesitando el encoder.

Como comprobación negativa, El proyecto final del paso no puede contener:

InMemoryUserDetailsManager

en código de producción.

Dos fuentes de usuario compitiendo harían ambiguo el mecanismo de autenticación y, sobre todo, dejarían vivo un estado pedagógico que 6.3 está diseñado para sustituir.

Paso 7 - Crear el inicializador de usuarios

Creamos UsuariosInicialesConfig.

El ejemplo utiliza un guard global `count() > 0`. Conservamos la explicación, pero el proyecto mejora el algoritmo a **find-or-create** para permitir evolución sin duplicados.

Roles materializados en **6.3**:

```

ADMIN
USER

```

Usuarios persistentes en **6.3**:

```

admin -> ADMIN + USER
ana   -> USER

```

El usuario gestor creado en memoria en 6.2 era deliberadamente temporal. Al eliminar InMemoryUserConfig en 6.3, esa identidad desaparece: **no** se persiste todavía ni se arrastra artificialmente al nuevo origen de datos. El rol y el usuario GESTOR reaparecerán de forma persistente en 6.8, cuando la arquitectura final los introduce.

El inicializador sigue siendo idempotente: reiniciar no crea duplicados de admin, ana, ADMIN o USER.

Paso 8 - Arrancar y verificar

Cuando el árbol M5 esté completamente materializado:

```
./mvnw spring-boot:run
```

En H2:

```
SELECT * FROM usuarios;  
SELECT * FROM roles;  
SELECT * FROM usuarios_roles;
```

La comprobación debe confirmar lo siguiente.

- usuarios contiene identidades persistentes;
- roles contiene nombres de rol;
- usuarios_roles contiene pares usuario_id/rol_id.

Pregunta de reflexión: ¿Por qué la tabla de unión no necesita guardar username ni nombre del rol?

Respuesta razonada: Porque sólo relaciona claves foráneas de usuarios y roles. Los atributos de cada entidad permanecen en su propia tabla y se obtienen mediante el join.

Paso 9 - Probar la autenticación

```
curl -i -u ana:ana123 \  
http://localhost:8080/api/v1/perfil
```

```
curl -i -u admin:admin123 \  
http://localhost:8080/api/v1/perfil
```

Comprobación negativa del lifecycle: el gestor in-memory de 6.2 ya no existe

```
curl -i -u gestor:gestor123 \  
http://localhost:8080/api/v1/gestor/documentos
```

Resultados esperados.

- ana → ROLE_USER;
- admin → ROLE_ADMIN, ROLE_USER;
- gestor:gestor123 → **401** en 6.3, porque la fuente in-memory se ha retirado y GESTOR aún no se ha persistido.

Evidencia conceptual. ana y admin conservan sus credenciales al cambiar el origen de identidades, mientras que el usuario temporal gestor demuestra que no estamos arrastrando datos futuros. Volverá a crearse persistentemente en 6.8.

Paso 10 - Crear el endpoint de registro

RegistroRequestDTO:

```
@NotBlank  
@Size(min = 3, max = 50)  
private String username;
```

```
@NotBlank  
@Size(min = 8)  
private String password;
```

```
@NotBlank
```

```
@Email
private String email;
```

UsuarioResponseDTO contiene:

```
id
username
email
roles
```

y **no contiene password**.

Pregunta de reflexión: ¿Por qué no devolvemos siquiera el hash de la contraseña?

Respuesta razonada: Porque el hash es material sensible que no aporta nada al cliente y puede facilitar ataques offline si se filtra. Los DTO de respuesta deben excluir por completo las credenciales.

Paso 11 - Crear el AuthService y el AuthController

AuthService.registrar(...):

1. rechaza username repetido;
2. rechaza email repetido;
3. exige que exista USER;
4. codifica contraseña;
5. asigna USER;
6. guarda;
7. proyecta a DTO.

El controlador expone:

POST /api/v1/auth/registro

y responde 201 Created.

Continuidad con M5. Se reutilizan las excepciones específicas ya existentes:

- RecursoDuplicadoException;
- RecursoNoEncontradoException;
- OperacionNoPermitidaException para el reto de contraseña.

No se vuelve a introducir la clase genérica NegocioException, que M5 conserva sólo como compatibilidad y marca @Deprecated.

Paso 12 - Configurar el endpoint público y probar

En SecurityConfig usamos un matcher mínimo:

```
.requestMatchers(HttpMethod.POST, "/api/v1/auth/registro").permitAll()
```

No abrimos todavía todo /api/v1/auth/**.

Prueba:

```
curl -i -X POST \
  http://localhost:8080/api/v1/auth/registro \
  -H "Content-Type: application/json" \
  -d '{
    "username": "pedro",
    "password": "pedro1234",
    "email": "pedro@educacion.gob.es"
  }'
```

La respuesta debe ser 201 Created y no debe contener la contraseña.

Después:

```
curl -i -u pedro:pedro1234 \
  http://localhost:8080/api/v1/perfil
```

Debe mostrar ROLE_USER.

Repite el registro con el mismo username o email.

El duplicado debe resolverse mediante el contrato de errores de negocio heredado de M5.

Paso 13 - Errores comunes del ejercicio

Error	Causa	Corrección
más de un UserDetailsService LazyInitializationException	no se cerró 6.2 roles fuera del contexto de persistencia	eliminar InMemoryUserConfig EAGER o carga explícita dentro de transacción
duplicado al registrar	username/email existentes	validar en servicio + constraint DB
contraseña visible en BD hasRole siempre falla	faltó encode authorities sin ROLE_	codificar antes de persistir transformar ADMIN → ROLE_ADMIN
registro devuelve 401	matcher no público	permitir sólo POST /api/v1/auth/registro
ciclo de beans	configuración acoplada	constructor injection y responsabilidades separadas
no aparece tabla intermedia roles no guardados usuario inexistente produce error técnico	mapping incompleto se modificó el lado incorrecto excepción incorrecta	revisar @JoinTable añadir al Set de Usuario lanzar UsernameNotFoundException

Comprobación adicional. Registrar no puede permitir que el cliente escoja ADMIN.

Paso 14 - Reto resuelto — Endpoint para cambiar la contraseña

El reto pide:

```
PUT /api/v1/perfil/password
```

y esa será la ruta canónica de nuestra solución.

DTO.

```
public class CambioPasswordRequestDTO {

    @NotBlank
    private String passwordActual;

    @NotBlank
    @Size(min = 8)
    private String passwordNueva;
}
```

Servicio.

```
@Transactional
public void cambiarPassword(
    String username,
    CambioPasswordRequestDTO request) {

    Usuario usuario = usuarioRepository.findByUsername(username)
```

```

        .orElseThrow(() ->
            new RecursoNoEncontradoException("Usuario", username));

    if (!passwordEncoder.matches(
        request.getPasswordActual(),
        usuario.getPassword())) {
        throw new OperacionNoPermitidaException(
            "La contraseña actual no es correcta");
    }

    usuario.setPassword(
        passwordEncoder.encode(request.getPasswordNueva()));
}

```

No es necesario llamar a save para una entidad gestionada dentro de la transacción, aunque hacerlo tampoco sería incorrecto.

Controlador.

El endpoint se añade al PerfilController:

```

@PutMapping("/password")
public ResponseEntity<Void> cambiarPassword(
    Authentication authentication,
    @Valid @RequestBody CambioPasswordRequestDTO request) {
    authService.cambiarPassword(authentication.getName(), request);
    return ResponseEntity.noContent().build();
}

```

Prueba.

```

curl -i -X PUT -u pedro:pedro1234 \
  http://localhost:8080/api/v1/perfil/password \
  -H "Content-Type: application/json" \
  -d '{
    "passwordActual": "pedro1234",
    "passwordNueva": "pedro5678"
  }'

```

La respuesta debe ser 204 No Content.

Después:

```

curl -i -u pedro:pedro5678 \
  http://localhost:8080/api/v1/perfil

```

La nueva contraseña funciona.

La antigua deja de funcionar:

```

curl -i -u pedro:pedro1234 \
  http://localhost:8080/api/v1/perfil

```

La petición debe devolver 401.

El snippet inicial coloca por error práctico el método bajo /api/v1/auth/password mientras el mismo punto abre /api/v1/auth/**. Eso podría convertir el cambio de contraseña en endpoint público. Nuestra solución conserva el **enunciado** del reto (/api/v1/perfil/password) y garantiza que permanezca autenticado.

Resultado esperado global

- Entidades Usuario y Rol con relación @ManyToMany.

- Repositorios persistentes y un `UsuarioDetailsService` que carga identidades desde la base de datos.
- Inicialización de roles/usuarios, registro público y cambio de contraseña.
- Contraseñas siempre codificadas y DTOs sin credenciales.

Resumen técnico del ejercicio

- `@ManyToMany` y `@JoinTable` entre `Usuario` y `Rol`.
- Carga persistente mediante `UserDetailsService` y `@Transactional`.
- Registro con contraseña codificada, unicidad y rol por defecto.
- Inicialización con `CommandLineRunner` y cambio seguro de contraseña.

Conclusión y enlace al siguiente punto

La autenticación ya depende de identidades persistentes y administrables. El siguiente punto utiliza esos usuarios y roles para profundizar en autorización por URL y por método.

Práctica 6.4 - Autorización por roles

Contexto del ejercicio: Vamos a aplicar autorización por URL y por método, crear operaciones administrativas, acceder a la identidad autenticada y probar los escenarios 401, 403 y 200.

Requisitos previos: Tener completado el punto 6.3 con usuarios y roles cargados desde la base de datos.

Paso 1 - Repasar el estado actual

Venimos de 6.3 con:

- usuarios persistentes;
- `UsuarioDetailsService`;
- roles persistentes `USER` y `ADMIN`; el gestor temporal de 6.2 ya no existe y `GESTOR` reaparecerá en 6.8;
- registro público específico;
- `/api/v1/admin/**` protegido por `ADMIN`;
- HTTP Basic todavía activo;
- CORS, OpenAPI y H2 siguen operativos.

Comprobación estructural:

```
grep -R "InMemoryUserDetailsService" src/main/java || true
```

No debe aparecer.

Pregunta de reflexión: ¿La regla `/api/v1/admin/**` protege una operación concreta o toda una zona del API?

Respuesta razonada: Protege cualquier petición cuyo path coincida con ese patrón. Para reglas específicas de una operación o de sus parámetros usamos además seguridad por método.

Paso 2 - Activar la seguridad por método

Evoluciona `SecurityConfig`:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    // ...
}
```

A partir de ahora Spring evalúa @PreAuthorize en beans interceptados.

Error crítico. Añadir @PreAuthorize sin @EnableMethodSecurity: la anotación queda sin el efecto esperado.

Comprueba que el estado intermedio compila antes de continuar:

```
./mvnw -B -DskipTests compile
```

Paso 3 - Crear un UsuarioController de administración

Creamos:

```
src/main/java/es/mecd/demo/miproyecto/auth/UsuarioController.java
```

```
@RestController
@RequestMapping("/api/v1/admin/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")
    public List<UsuarioResponseDTO> listar() {
        return usuarioService.listarTodos();
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public UsuarioResponseDTO consultar(@PathVariable Long id) {
        return usuarioService.consultar(id);
    }
}
```

Doble barrera.

1. /api/v1/admin/** exige ADMIN en SecurityFilterChain.
2. Los métodos vuelven a exigir ADMIN.

Pregunta de reflexión: Si la URL ya exige ADMIN, ¿qué protege la anotación ante un futuro cambio de ruta?

Respuesta razonada: La regla de URL protege la zona administrativa, pero @PreAuthorize mantiene la intención de seguridad unida al método. Si mañana cambia la ruta durante un refactor, el método sigue exigiendo ADMIN y evita que la operación quede abierta por accidente.

Comprueba que el estado intermedio compila antes de continuar:

```
./mvnw -B -DskipTests compile
```

Paso 4 - Crear el UsuarioService

El servicio necesita UsuarioRepository para leer usuarios y RolRepository para el reto de cambio de roles.

```
@Service
public class UsuarioService {
    // ...
}
```

```

@Transactional(readOnly = true)
public List<UsuarioResponseDTO> listarTodos() {
    return usuarioRepository.findAll().stream()
        .map(this::toDTO)
        .toList();
}

@Transactional(readOnly = true)
public UsuarioResponseDTO consultar(Long id) {
    return usuarioRepository.findById(id)
        .map(this::toDTO)
        .orElseThrow(() ->
            new RecursoNoEncontradoException("Usuario", id));
}
}

```

El mapeo ordena los nombres de rol para producir respuestas deterministas.

Pregunta de reflexión: ¿Podría utilizarse `@PreAuthorize` también en un método del servicio?

Respuesta razonada: Sí. Colocarla en el servicio protege también invocaciones que no entren por ese controlador. Es útil cuando la regla pertenece al caso de uso y no sólo al transporte HTTP.

Comprueba que el estado intermedio compila antes de continuar:

```
./mvnw -B -DskipTests compile
```

Paso 5 - Añadir reglas de URL y de método combinadas

Ahora combinamos las dos capas con una cadena completa del estado 6.4. Conservamos HTTP Basic porque todavía es el mecanismo activo y mantenemos **sólo** el registro como operación pública de `/auth`:

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .cors(Customizer.withDefaults())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.POST, "/api/v1/auth/registro").permitAll()
            .requestMatchers("/api/v1/public/**").permitAll()
            .requestMatchers("/h2-console/**").permitAll()
            .requestMatchers("/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
            .requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR", "ADMIN")
            .anyRequest().authenticated()
        )
        .csrf(csrf -> csrf.disable())
        .httpBasic(Customizer.withDefaults());

    return http.build();
}

```

La regla de URL protege la zona administrativa antes de llegar al controlador, y el controlador añade además:

```
@PreAuthorize("hasRole('ADMIN')")
```

Un USER puede ser rechazado por la cadena antes de que el controlador llegue siquiera a evaluarse.

Ejecuta la suite después de combinar seguridad por URL y por método:

```
./mvnw -B test
```

Pregunta de reflexión: ¿Qué pasa si un usuario con rol USER intenta acceder a `/api/v1/admin/usuarios`? ¿Se evalúa la anotación?

Respuesta razonada: La regla de URL lo rechaza antes de que llegue al método, por lo que en esa petición concreta `@PreAuthorize` no necesita decidir nada. La anotación sigue siendo útil como segunda barrera y protege la operación si la ruta cambia o si el método se invoca por otra vía gestionada por Spring.

Paso 6 - Probar la autorización con distintos usuarios

```
# USER autenticado
curl -i -u ana:ana123 \
  http://localhost:8080/api/v1/admin/usuarios
```

```
# ADMIN
curl -i -u admin:admin123 \
  http://localhost:8080/api/v1/admin/usuarios
```

```
# Anónimo
curl -i \
  http://localhost:8080/api/v1/admin/usuarios
```

Esperado:

```
ana      -> 403
admin    -> 200
anónimo  -> 401
```

Prueba los tres escenarios de autorización:

```
curl -i http://localhost:8080/api/v1/admin/usuarios
curl -i -u ana:ana123 http://localhost:8080/api/v1/admin/usuarios
curl -i -u admin:admin123 http://localhost:8080/api/v1/admin/usuarios
```

Pregunta de reflexión: ¿Qué diferencia hay entre el 403 de Ana y un 401?

Respuesta razonada: Ana ya se ha autenticado correctamente, pero no tiene el rol ADMIN, por eso recibe 403. Un 401 indicaría que falta una identidad válida o que las credenciales no han sido aceptadas.

Paso 7 - Crear un endpoint para consultar el propio perfil

El perfil ya existe desde 6.2/6.3. Ajústalo para trabajar con el principal autenticado:

```
@GetMapping
public Map<String, Object> perfil(
    @AuthenticationPrincipal UserDetails user) {
    return Map.of(
        "usuario", user.getUsername(),
        "authorities", user.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .sorted()
            .toList());
}
```

El cambio de contraseña conserva Authentication, porque en ese método sólo necesitamos `getName()` y su contrato ya está probado.

`@AuthenticationPrincipal` entrega el `UserDetails` que Spring colocó en el contexto.

Comprueba el perfil autenticado:

```
curl -i -u ana:anal23 http://localhost:8080/api/v1/perfil
```

Pregunta de reflexión: ¿Qué diferencia hay entre `@AuthenticationPrincipal` y `Authentication`?

Respuesta razonada: `Authentication` representa el objeto completo de autenticación, incluidas las autoridades y el principal. `@AuthenticationPrincipal` inyecta directamente ese principal ya tipado, lo que resulta más cómodo cuando sólo necesitamos los datos del usuario autenticado.

Paso 8 - Crear el `UsuarioPrincipal` (para el punto 6.7)

Creemos ahora, sin integrarlo todavía:

```
public class UsuarioPrincipal implements UserDetails {
    private final Long id;
    private final String username;
    private final String email;
    private final Collection<? extends GrantedAuthority> authorities;

    // constructor + getters + contrato UserDetails
}
```

`getPassword()` devuelve null: este principal está diseñado para representar una identidad ya validada desde token en 6.7, no para almacenar credenciales.

Preparación para el filtro JWT.

- La clase existe.
- `UsuarioDetailsService` sigue devolviendo `User` estándar.
- La integración se cierra en **6.7**.

Comprueba que el estado intermedio compila antes de continuar:

```
./mvnw -B -DskipTests compile
```

Pregunta de reflexión: ¿Por qué no podemos usar esta clase en el `UsuarioDetailsService` todavía?

Respuesta razonada: Porque en 6.4 estamos preparando `UsuarioPrincipal` para el flujo JWT posterior, pero la autenticación activa sigue siendo usuario/contraseña y el `UsuarioDetailsService` continúa devolviendo el `User` estándar. La sustitución se realiza cuando el filtro JWT construya el principal a partir de claims verificados en 6.7.

Paso 9 - Probar la expresión de autorización

El ejercicio propone la expresión futura:

```
#id == authentication.principal.id
```

pero también advierte que **todavía no puede funcionar** con el principal actual, porque `User` de Spring Security no tiene `getId()`.

Por tanto este paso se ejecuta como **prueba negativa controlada**, no como código final roto.

Si se añade temporalmente una expresión que accede a:

```
authentication.principal.id
```

la evaluación puede lanzar `SpelEvaluationException`.

Restauración obligatoria. Elimina la expresión temporal antes de continuar. No queda ningún endpoint final de 6.4 dependiendo de `principal.id`.

El caso funcional se resolverá en 6.7, donde el principal será `UsuarioPrincipal`.

Pregunta de reflexión: ¿Qué error daría la expresión `#id == authentication.principal.id` si el principal fuera un `String`?

Respuesta razonada: La expresión no podría resolver la propiedad id y Spring Expression Language lanzaría un error de evaluación. Por eso la expresión sólo debe quedar activa cuando el principal real sea un objeto que exponga getId().

Paso 10 - Escribir tests de autorización

Este es el **primer punto del recorrido** en el que usamos utilidades de spring-security-test, concretamente SecurityMockMvcRequestPostProcessors.user(...). Por tanto la dependencia debe incorporarse **ahora**, en 6.4, aunque la fuente la presente formalmente de nuevo en 6.9. Si se esperara hasta 6.9, este snapshot no podría compilar sus propios tests.

Añade al pom.xml:

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

Comprueba inmediatamente el classpath de test:

```
./mvnw -B -DskipTests test-compile
```

En Spring Boot 3.5 utiliza @MockitoBean para declarar el mock dentro del contexto del test.

```
@WebMvcTest(UsuarioController.class)
@Import(SecurityConfig.class)
class UsuarioControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private UsuarioService usuarioService;

    @Test
    void listar_debeDevolver403_cuandoNoEsAdmin()
        throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios")
            .with(user("ana").roles("USER")))
            .andExpect(status().isForbidden());
    }

    @Test
    void listar_debeDevolver200_cuandoEsAdmin()
        throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios")
            .with(user("admin").roles("ADMIN")))
            .andExpect(status().isOk());
    }

    @Test
    void listar_debeDevolver401_cuandoNoAutenticado()
        throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios")
            .andExpect(status().isUnauthorized());
    }
}
```

Con estos tests cubrimos Cubrir las tres ramas semánticamente distintas: 401, 403 y 200.

Ejecuta únicamente los tests de autorización mientras trabajas en este punto:

```
./mvnw -B -Dtest=UsuarioControllerTest test
```

Pregunta de reflexión: ¿Por qué es importante testear los tres escenarios?

Respuesta razonada: Porque 401, 403 y 200 demuestran contratos distintos: ausencia de autenticación, autenticación sin permiso y acceso autorizado. Probar sólo el caso de éxito no detectaría rutas abiertas accidentalmente ni reglas demasiado permisivas.

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
@PreAuthorize no se aplica hasRole("ROLE_ADMIN") falla hasAuthority("ADMIN") falla SpEL no resuelve id 403 con rol esperado	falta @EnableMethodSecurity hasRole gestiona el prefijo authority real es ROLE_ADMIN principal actual no expone id transformación ROLE_ inconsistente	activarlo usar hasRole("ADMIN") usar literal correcto esperar a UsuarioPrincipal revisar UsuarioDetailsService
endpoint anónimo da 200	falta regla general	mantener anyRequest().authenticated() proteger método público
anotación en privado	proxy no intercepta como se espera	
llamada interna elude seguridad	self-invocation	mover frontera a otro bean
test usa API obsoleta test 401/403 confundido	@MockBean antiguo identidad/rol mal preparados	usar @MockitoBean separar escenario anónimo y autenticado

Cuando hayas corregido un error, ejecuta de nuevo la suite para evitar que la corrección abra otra ruta:

```
./mvnw -B test
```

Paso 12 - Reto resuelto — Endpoint de cambio de rol

El reto exige:

PUT /api/v1/admin/usuarios/{id}/roles

DTO.

```
public class CambioRolesRequestDTO {
    @NotEmpty(message = "Debe indicar al menos un rol")
    private List<String> roles;

    // getter/setter
}
```

Servicio.

```
@Transactional
public UsuarioResponseDTO cambiarRoles(
    Long id,
    List<String> nombresRoles) {

    Usuario usuario = usuarioRepository.findById(id)
        .orElseThrow(() ->
            new RecursoNoEncontradoException(
```

```

        "Usuario", id));

    Set<Rol> roles = new HashSet<>();
    for (String nombre : nombresRoles) {
        Rol rol = rolRepository.findByNombre(nombre)
            .orElseThrow(() ->
                new RecursoNoEncontradoException(
                    "Rol", nombre));
        roles.add(rol);
    }

    usuario.setRoles(roles);
    return toDTO(usuario);
}

```

Dentro de la transacción no necesitamos un save redundante para una entidad gestionada; dejamos que el dirty checking persista el cambio.

Controlador.

```

@PutMapping("/{id}/roles")
@PreAuthorize("hasRole('ADMIN')")
public UsuarioResponseDTO cambiarRoles(
    @PathVariable Long id,
    @Valid @RequestBody CambioRolesRequestDTO request) {
    return usuarioService.cambiarRoles(
        id,
        request.getRoles());
}

```

En el estado acumulativo de 6.4 sólo están persistidos los roles USER y ADMIN. El usuario y el rol GESTOR que existieron temporalmente en memoria en 6.2 no se materializan en base de datos hasta 6.8. Por tanto, las pruebas de este snapshot deben usar únicamente roles que realmente existen en 6.4.

Prueba negativa primero. Ana (id=2) todavía tiene sólo USER, así que comprobamos el 403 antes de modificar sus roles:

```

curl -i -X PUT -u ana:ana123 \
  http://localhost:8080/api/v1/admin/usuarios/2/roles \
  -H "Content-Type: application/json" \
  -d '{"roles":["ADMIN"]}'

```

La petición debe devolver 403 Forbidden y no debe modificar los roles del usuario.

Prueba positiva. Ahora el administrador asigna al usuario 2 los dos roles que existen en este snapshot:

```

curl -i -X PUT -u admin:admin123 \
  http://localhost:8080/api/v1/admin/usuarios/2/roles \
  -H "Content-Type: application/json" \
  -d '{"roles":["USER","ADMIN"]}'

```

La respuesta debe ser 200 OK y el usuario debe quedar con ADMIN y USER (la respuesta puede mostrarlos ordenados). GESTOR se probará de forma persistente a partir de 6.8.

Pregunta de reflexión: ¿Por qué el endpoint requiere rol ADMIN? ¿Qué pasaría si un usuario normal pudiera cambiar sus propios roles?

Respuesta razonada: Cambiar roles modifica privilegios. Si un usuario ordinario pudiera asignárselos a sí mismo, podría elevar sus permisos hasta ADMIN o GESTOR y romper completamente el modelo de autorización. Por eso la operación debe quedar restringida a una autoridad administrativa.

Resultado esperado global

- @EnableMethodSecurity activo.
- Administración de usuarios protegida por ADMIN con reglas URL + @PreAuthorize.
- UsuarioPrincipal preparado para la fase JWT y endpoints con @AuthenticationPrincipal.
- Tests que distinguen 401, 403 y acceso autorizado.

Resumen técnico del ejercicio

- Autorización por URL y por método.
- hasRole, hasAuthority, hasAnyRole y SpEL.
- @EnableMethodSecurity, @PreAuthorize, @AuthenticationPrincipal.
- Principal personalizado y tests de autorización.

Conclusión y enlace al siguiente punto

La autorización queda expresada en dos capas complementarias: reglas generales por URL y reglas contextuales por método. En 6.5 cambiaremos de foco para estudiar qué es un JWT antes de implementarlo.

Práctica 6.5 - Introducción a JWT

Contexto del ejercicio: Vamos a estudiar la estructura de JWT generando y verificando tokens manualmente con Java estándar, para entender header, payload, firma, expiración e integridad antes de utilizar una librería.

Requisitos previos: Tener completados los puntos anteriores de autenticación y autorización. Este punto es un laboratorio didáctico y todavía no integra JWT en Spring Security.

Paso 1 - Entender la estructura con un ejemplo

El ejercicio propone inspeccionar este JWT didáctico:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbmEiLCJyb2xlcyI6WyJST0xFX1VTRVliXSwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

Sin los saltos de línea forma una sola cadena.

Header:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload:

```
{
  "sub": "ana",
  "roles": ["ROLE_USER"],
  "iat": 1516239022
}
```

Header y payload son legibles sin conocer la clave.

El ejemplo usa jwt.io para visualizarlo. Sólo se deben pegar allí **tokens didácticos**, nunca access/refresh tokens reales de un sistema.

Pregunta de reflexión: Si el payload se puede leer, ¿qué aporta entonces la firma?

Respuesta razonada: La firma permite detectar modificaciones y comprobar que el token fue firmado con la clave esperada. No oculta el payload: aporta integridad y autenticidad, no confidencialidad.

Paso 2 - Crear una clase para generar un JWT manualmente

Creamos en código de pruebas:

```
src/test/java/es/meqd/demo/miproyecto/jwt/JwtManual.java
```

La clase usa exclusivamente Java estándar:

```
Base64.getUrlEncoder().withoutPadding()
Mac.getInstance("HmacSHA256")
SecretKeySpec
```

Flujo:

```
JSON header
  ↓ Base64Url
header64
```

```
JSON payload
  ↓ Base64Url
payload64
```

```
header64.payload64
  ↓ HMAC-SHA256(secret)
signature64
```

```
header64.payload64.signature64
```

La constante incluida en la clase está marcada DIDACTIC_SECRET: es una clave de ejemplo y **no** un secreto de producción.

Experimento JWT manual. Firma JWT manual sólo para aprendizaje. La implementación operativa se reemplaza por JJWT en 6.6.

Paso 3 - Decodificar el token generado

La utilidad incorpora:

```
public static String decodificarPayload(String jwt) {
    String[] partes = jwt.split("\\.", -1);
    if (partes.length != 3) {
        throw new IllegalArgumentException("Token mal formado");
    }

    byte[] decoded = Base64.getUrlDecoder()
        .decode(partes[1]);
    return new String(decoded, StandardCharsets.UTF_8);
}
```

Ejecuta **primero JwtManual.main()** desde el IDE. Esta ejecución es parte obligatoria del ejercicio: el test unitario no sustituye el observable manual. Debes ver una salida equivalente a:

```
JWT generado:
eyJ...
Token válido: true
Token modificado válido: false
Con clave incorrecta: false
Payload decodificado: {"sub":"ana",...}
```

Después ejecuta también el test automatizado:

```
./mvnw -Dtest=JwtManualTest test
```

El test comprueba la firma, la manipulación real del payload, la clave incorrecta y que `main()` produce los observables anteriores.

Pregunta de reflexión: ¿Ha sido necesaria la clave para leerlo?

Respuesta razonada: No. Base64Url es codificación, no cifrado. La clave sólo es necesaria para producir o verificar la firma según el algoritmo utilizado.

Paso 4 - Modificar el payload sin actualizar la firma

Toma un token válido y sustituye sólo su segunda parte por otro payload Base64Url, conservando la firma antigua. En el proyecto se hace con `JwtManual.adulterarPayload(...)`. No uses `jwt.replace("ana", "pedro")`: el texto ana no aparece literalmente en el JWT porque el payload está codificado en Base64Url.

Ejemplo conceptual:

```
header.payload_original.firma_original
      ↓ cambiar
header.payload_modificado.firma_original
```

La decodificación sigue funcionando, pero la verificación criptográfica falla.

Esto demuestra por qué:

payload legible ≠ payload confiable

Restauración. El token manipulado sólo se usa como dato de prueba y no se reutiliza en pasos posteriores como token válido.

Paso 5 - Verificar el token con Java

`JwtManual.verificar(jwt, secret):`

1. exige exactamente tres partes;
2. vuelve a firmar `header.payload`;
3. decodifica firma esperada y recibida;
4. compara los bytes con `MessageDigest.isEqual`.

Ejemplo:

```
boolean valido = JwtManual.verificar(
    jwt,
    JwtManual.DIDACTIC_SECRET);
```

Token intacto → true; token manipulado → false.

Importante. Esta verificación manual valida la **firma**, no interpreta todavía exp. La expiración real será validada por la biblioteca en 6.6.

Paso 6 - Probar con distintos payloads

El ejercicio amplía los claims:

```
{
  "sub": "ana",
  "email": "ana@educacion.gob.es",
  "roles": ["ROLE_USER", "ROLE_GESTOR"],
  "iat": 1516239022,
  "exp": 9999999999
}
```

Genera el token y decodifícalo localmente.

Resultados esperados.

- sub, iat y exp son claims registrados;
- email y roles son datos de aplicación en este ejemplo;
- añadir claims aumenta el tamaño del token;
- el contenido sigue siendo legible.

Pregunta de reflexión: ¿Qué dato de esa lista no debería añadirse si no es necesario para tomar decisiones?

Respuesta razonada: Cualquier dato personal o sensible que el servidor no necesite para autenticar/autorizar. Los claims deben reducirse al mínimo porque el payload es legible por quien posea el token.

Paso 7 - Reflexionar sobre la seguridad

Responde y justifica:

1. ¿Se puede leer el payload sin la clave? **Sí**, porque Base64Url no cifra.
2. ¿Se puede cambiar sin invalidar la firma? **No**, salvo que el atacante pueda crear una firma válida.
3. ¿Qué ocurre si se filtra una clave HS256? Un atacante podría emitir tokens que el servidor aceptaría como firmados por nosotros.
4. ¿Debe viajar una contraseña dentro del payload? **No**.
5. ¿Un token firmado necesita HTTPS? **Sí**: la firma no impide que otro capture y reutilice el token.
6. ¿Debemos registrar tokens completos en logs? **No**.

El modelo de amenaza queda explícito antes de introducir la librería.

Paso 8 - Comparar con un token real

En 6.6 JWT hará de forma robusta lo que aquí hemos hecho manualmente:

- construir/serializar claims;
- codificar las partes;
- firmar;
- parsear;
- verificar firma;
- verificar expiración;
- convertir errores en excepciones específicas.

La estructura compacta seguirá siendo:

header.payload.signature

Cierre conceptual. Comprender la implementación manual sirve para diagnosticar; no justifica reimplementar JWT en código productivo.

Paso 9 - Errores comunes del ejercicio

Error	Causa	Corrección
IllegalArgumentExcepcion Base64 firma no coincide aparece = al final no hay tres partes	alfabeto/formato incorrecto clave o contenido distintos encoder estándar/padding token mal formado	usar getUrlEncoder/getUrlDecoder verificar header.payload y clave withoutPadding() validar estructura antes de acceder
NoSuchAlgorithmException caracteres extraños	nombre de algoritmo incorrecto charset implícito	HmacSHA256 usar UTF-8

Error	Causa	Corrección
“decodifica, luego es válido”	confundir lectura con verificación	verificar firma
token real enviado a web externa	exposición de credencial	usar tokens de ejemplo o decoder local

Paso 10 - Experimentar con la expiración

Genera un payload:

```
{
  "sub": "ana",
  "exp": 1
}
```

Al decodificarlo se ve que exp representa un instante muy antiguo.

Es importante observar que Nuestra función manual verificar sólo comprueba HMAC y, por diseño pedagógico, podría devolver true aunque exp esté vencido.

Esto no contradice JWT: demuestra que el consumidor debe validar **firma + claims temporales**. JWT se encargará de esa política en 6.6.

Pregunta de reflexión: ¿Qué peligro tendría verificar únicamente la firma?

Respuesta razonada: Aceptaríamos tokens firmados pero expirados o de tipo incorrecto. La validación debe incluir firma, tiempos, tipo de token y los claims exigidos por el contrato.

Paso 11 - Verificar el token con una clave incorrecta

```
boolean valido = JwtManual.verificar(
    jwt,
    "otraClaveQueNoEsLaOriginal");
```

El resultado debe ser false.

Con HS256, la capacidad de generar una firma válida depende del secreto compartido.

Pregunta de reflexión: ¿Por qué una fuga de esa clave es más grave que la exposición del payload?

Respuesta razonada: Porque con una clave simétrica robada un atacante puede fabricar tokens con firmas válidas y suplantar identidades. Leer un payload expone datos, pero no permite por sí solo firmar un token nuevo.

Paso 12 - Reto resuelto — Decodificar un token sin verificar la firma

El método queda implementado en JwtManual:

```
public static String decodificarPayload(String jwt) {
    String[] partes = jwt.split("\\.", -1);
    if (partes.length != 3) {
        throw new IllegalArgumentException("Token mal formado");
    }
    byte[] decoded = Base64.getUrlDecoder()
        .decode(partes[1]);
    return new String(decoded, StandardCharsets.UTF_8);
}
```

Uso:

```
String payload = JwtManual.decodificarPayload(jwt);  
System.out.println(payload);
```

La regla que debe quedar clara es:

DECODIFICAR = inspeccionar

VERIFICAR = decidir si podemos confiar

Nunca se debe autorizar una operación basándose únicamente en un payload decodificado.

Resultado esperado global

- Comprensión práctica de header, payload y signature.
- Generación y verificación manual HMAC-SHA256.
- Comprobación de que manipular el payload invalida la firma.
- Decodificación del payload sin confundir lectura con verificación.

Resumen técnico del ejercicio

- Base64Url para header/payload y firma HMAC-SHA256.
- Claims y expiración.
- Integridad frente a manipulación y diferencia entre decodificar/verificar.
- Casos de uso, límites y buenas prácticas de JWT.

Conclusión y enlace al siguiente punto

Ya sabemos leer, firmar y verificar manualmente un JWT y también sus límites. En 6.6 sustituiremos el código criptográfico manual por JJWT y construiremos tokens reales de la aplicación.

Práctica 6.6 - Generación de tokens JWT

Contexto del ejercicio: Vamos a añadir JJWT al proyecto, configurar la clave secreta, crear un Jwt-Service que genere tokens y añadir endpoints de login y refresh. También resolveremos el reto de logout.

Requisitos previos: Tener el proyecto con usuarios en base de datos y autorización del punto 6.4, y haber comprendido la estructura de JWT en 6.5.

Paso 1 - Añadir las dependencias de JJWT

Archivo: pom.xml.

Usa JJWT **0.12.3**: jjwt-api en compile y jjwt-impl/jjwt-jackson en runtime.

```
./mvnw -B -DskipTests compile
```

Maven debe resolver los tres módulos y el código debe poder importar Jwts, Claims, Decoders y Keys.

Si falta uno de los módulos necesarios en runtime, la compilación puede llegar a pasar, pero el parseo o la serialización fallarán al ejecutar. Por eso conservamos los tres módulos indicados.

Añade los tres módulos de JJWT. Usamos exactamente la versión **0.12.3** indicada en el PDF original, manteniendo la separación entre API y módulos de runtime:

```
<dependency>  
  <groupId>io.jsonwebtoken</groupId>  
  <artifactId>jjwt-api</artifactId>  
  <version>0.12.3</version>  
</dependency>  
<dependency>  
  <groupId>io.jsonwebtoken</groupId>
```

```

<artifactId>jjwt-impl</artifactId>
<version>0.12.3</version>
<scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.3</version>
  <scope>runtime</scope>
</dependency>

```

Pregunta de reflexión: ¿Por qué jjwt-impl y jjwt-jackson tienen scope=runtime?

Respuesta razonada: Porque nuestra aplicación compila contra la API pública de jjwt-api. La implementación concreta y la integración con Jackson se necesitan al ejecutar, pero no deben convertirse en APIs que nuestro código importe directamente.

Paso 2 - Configurar las propiedades de JWT

M6 añade jwt.secret, jwt.expiration y jwt.refresh-expiration a dev/test/prod. Dev y test usan secretos Base64 no productivos; producción exige \${JWT_SECRET}.

```

jwt.secret=${JWT_SECRET}
jwt.expiration=${JWT_EXPIRATION:3600000}
jwt.refresh-expiration=${JWT_REFRESH_EXPIRATION:604800000}

```

En el perfil de producción no debe existir un secreto embebido; si JWT_SECRET falta o no contiene una clave Base64 válida y suficientemente fuerte, el arranque debe fallar.

En este proyecto, si JwtConfig decodifica Base64, el fallback de desarrollo debe ser Base64 real. M6 elimina la contradicción “texto cualquiera + Decoders.BASE64”.

Mantén propiedades coherentes en los perfiles. Por ejemplo, desarrollo y test pueden usar secretos didácticos distintos, mientras producción exige una variable de entorno:

```

# application-dev.properties
jwt.secret=${JWT_SECRET_DEV:<BASE64_DE_DESARROLLO>}
jwt.expiration=3600000
jwt.refresh-expiration=604800000

# application-test.properties
jwt.secret=<BASE64_EXCLUSIVO_DE_TEST>
jwt.expiration=3600000
jwt.refresh-expiration=604800000

# application-prod.properties
jwt.secret=${JWT_SECRET}
jwt.expiration=${JWT_EXPIRATION:3600000}
jwt.refresh-expiration=${JWT_REFRESH_EXPIRATION:604800000}

```

Comprueba dónde se declaran:

```
grep -R "^jwt\." src/main/resources/application-*.properties
```

Pregunta de reflexión: ¿Por qué hay un valor por defecto para jwt.secret?

Respuesta razonada: Sólo para facilitar el arranque en desarrollo y ejercicios locales. En producción el secreto debe proporcionarse desde una variable de entorno o un gestor de secretos; dejar un valor conocido en el código o en el repositorio haría predecible la clave de firma.

Paso 3 - Crear la configuración de la clave

Archivo: auth/JwtConfig.java.

```

@Bean
SecretKey jwtSecretKey() {
    return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
}

```

La implementación captura errores de formato y los convierte en un fallo de configuración explícito.

Una clave de menos de 256 bits o un Base64 inválido debe impedir el arranque en lugar de permitir que el sistema funcione con una clave débil.

La configuración de la clave queda centralizada en un bean:

```

@Configuration
public class JwtConfig {
    @Bean
    SecretKey jwtSecretKey(@Value("${jwt.secret}") String secret) {
        byte[] bytes = Decoders.BASE64.decode(secret);
        return Keys.hmacShaKeyFor(bytes);
    }
}

```

Después de crearla:

```
./mvnw -B -DskipTests compile
```

Pregunta de reflexión: ¿Qué pasa si `jwt.secret` no está bien formado en Base64?

Respuesta razonada: La decodificación falla al construir la `SecretKey` y la aplicación no debería arrancar con una clave inválida. Es preferible fallar al inicio que descubrir el problema cuando se intenta firmar o verificar un token.

Paso 4 - Crear el JwtService

Archivo: `auth/JwtService.java`.

Implementa:

- `generarToken(Usuario);`
- `generarRefreshToken(Usuario);`
- `extraerUsername / extraerId / extraerClaims;`
- `extraerAuthorities;`
- `esValido;`
- `esAccessTokenValido;`
- `esRefreshTokenValido.`

El access contiene `tipo=access`, `jti`, `id/email/roles`; refresh contiene `tipo=refresh` y `jti`, sin roles.

```

Claims claims = Jwts.parser()
    .verifyWith(clave)
    .build()
    .parseSignedClaims(token)
    .getPayload();

```

A partir de aquí JWT es la implementación operativa; `JwtManual` queda únicamente como ejercicio didáctico y apoyo de test.

Si aparece este error, un claim numérico puede llegar como cualquier `Number`; M6 convierte con `long-Value()` y no asume que siempre sea `Long`.

El servicio real conserva en una sola clase la generación y la validación. La parte esencial es ésta:

```

@Service
public class JwtService {
    private static final String CLAIM_TIPO = "tipo";
    private static final String TIPO_ACCESS = "access";
}

```

```

private static final String TIPO_REFRESH = "refresh";

private final SecretKey clave;
private final long expirationMs;
private final long refreshExpirationMs;

public JwtService(
    SecretKey clave,
    @Value("${jwt.expiration}") long expirationMs,
    @Value("${jwt.refresh-expiration}") long refreshExpirationMs) {
    this.clave = clave;
    this.expirationMs = expirationMs;
    this.refreshExpirationMs = refreshExpirationMs;
}

public String generarToken(Usuario usuario) {
    Date ahora = new Date();
    Date exp = new Date(ahora.getTime() + expirationMs);
    List<String> roles = usuario.getRoles().stream()
        .map(r -> "ROLE_" + r.getNombre())
        .sorted()
        .toList();
    return Jwts.builder()
        .subject(usuario.getUsername())
        .id(UUID.randomUUID().toString())
        .claim(CLAIM_TIPO, TIPO_ACCESS)
        .claim("id", usuario.getId())
        .claim("email", usuario.getEmail())
        .claim("roles", roles)
        .issuedAt(ahora)
        .expiration(exp)
        .signWith(clave, Jwts.SIG.HS256)
        .compact();
}

public Claims extraerClaims(String token) {
    return Jwts.parser()
        .verifyWith(clave)
        .build()
        .parseSignedClaims(token)
        .getPayload();
}
}

```

Comprueba que el servicio y sus dependencias compilan antes de continuar. En 6.6 todavía no introducimos tests de integración JWT nuevos; éstos se incorporarán en los puntos de testing que corresponden:

```
./mvnw -B -DskipTests compile
```

Pregunta de reflexión: ¿Qué diferencia hay entre `parseSignedClaims` y `parseClaimsJwt`?

Respuesta razonada: `parseSignedClaims` procesa un JWT firmado y verifica su firma antes de devolver los claims. Un método destinado a JWT no firmados no ofrece esa garantía y no debe usarse para tomar decisiones de autenticación o autorización.

Paso 5 - Crear los DTOs de login

Archivos:

- LoginRequestDTO;
- LoginResponseDTO;
- RefreshRequestDTO.

LoginResponseDTO devuelve snake_case con @JsonProperty y expires_in en segundos.

```
{
  "access_token": "...",
  "refresh_token": "...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

Si aparece este error, no devolver expires_in=3600000; convertir ms → s.

Los tres DTO separan entrada, salida y renovación:

```
public class LoginRequestDTO {
    @NotBlank
    private String username;
    @NotBlank
    private String password;
    // getters y setters
}

public class LoginResponseDTO {
    @JsonProperty("access_token")
    private String accessToken;
    @JsonProperty("refresh_token")
    private String refreshToken;
    @JsonProperty("token_type")
    private String tokenType = "Bearer";
    @JsonProperty("expires_in")
    private long expiresIn;
    // constructor y getters
}

public class RefreshRequestDTO {
    @NotBlank(message = "El refresh token es obligatorio")
    @JsonProperty("refresh_token")
    private String refreshToken;
    // getter y setter
}
```

Pregunta de reflexión: ¿Por qué la respuesta usa nombres JSON como access_token y refresh_token aunque los campos Java estén en camelCase?

Respuesta razonada: Porque el contrato HTTP puede seguir una convención independiente de los nombres internos de Java. @JsonProperty permite mantener un JSON estable en snake_case sin degradar la legibilidad del código Java.

Paso 6 - Ampliar el AuthService con login y refresh

AuthService.login usa AuthenticationManager; no compara passwords manualmente. Captura AuthenticationException y devuelve una excepción funcional 401 sin enumerar usuarios.

refresh exige:

1. firma/expiración válidas;
2. tipo=refresh;
3. jti no consumido;
4. usuario aún existente;
5. generar un nuevo access y **nuevo refresh**.

TokenRevocationService.consumeRefresh hace que el refresh anterior sea de un solo uso dentro del proceso.

En este proyecto, jwtService.esValido() por sí solo permitiría usar un access token como refresh. M6 separa tipos y lo rechaza.

```

        usuario.setPassword(passwordEncoder.encode(request.getPassword()));
        usuario.setEmail(request.getEmail());
        usuario.setActivo(true);
        usuario.getRoles().add(rolUser);
        return toDTO(usuarioRepository.save(usuario));
    }

    @Transactional(readOnly = true)
    public LoginResponseDTO login(LoginRequestDTO request) {
        try {
            authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(
                    request.getUsername(), request.getPassword());
        } catch (AuthenticationException ex) {
            throw new CredencialesInvalidasException();
        }

        Usuario usuario = usuarioRepository.findByUsername(request.getUsername())
            .orElseThrow(CredencialesInvalidasException::new);
        return crearPar(usuario);
    }

    @Transactional(readOnly = true)
    public LoginResponseDTO refresh(String refreshToken) {
        if (!jwtService.esRefreshTokenValido(refreshToken)) {
            throw new TokenInvalidoException("Refresh token inválido o expirado");
        }

        Claims claims = jwtService.extraerClaims(refreshToken);
        String jti = claims.getId();
        if (jti == null || !tokenRevocationService.consumeRefresh(
            jti, claims.getExpiration().toInstant())) {
            throw new TokenInvalidoException("Refresh token ya utilizado");
        }

        String username = claims.getSubject();
        Usuario usuario = usuarioRepository.findByUsername(username)
            .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", username));
        if (!usuario.isActivo()) {
            throw new TokenInvalidoException("Usuario inactivo");
        }
        return crearPar(usuario);
    }

    public void logout(String accessToken) {
        if (!jwtService.esAccessTokenValido(accessToken)) {
            throw new TokenInvalidoException("Access token inválido o expirado");
        }

        Claims claims = jwtService.extraerClaims(accessToken);
        tokenRevocationService.revokeAccess(
            claims.getId(), claims.getExpiration().toInstant());
    }

```

```

    }

    @Transactional
    public void cambiarPassword(
        String username,
        CambioPasswordRequestDTO request) {
        Usuario usuario = usuarioRepository.findByUsername(username)
            .orElseThrow(() -> new RecursoNoEncontradoException("Usuario", username));

        if (!passwordEncoder.matches(
            request.getPasswordActual(), usuario.getPassword())) {
            throw new OperacionNoPermitidaException(
                "La contraseña actual no es correcta");
        }

        usuario.setPassword(passwordEncoder.encode(request.getPasswordNueva()));
    }

    private LoginResponseDTO crearPar(Usuario usuario) {
        return new LoginResponseDTO(
            jwtService.generarToken(usuario),
            jwtService.generarRefreshToken(usuario),
            jwtService.getExpirationMs() / 1000);
    }

    private UsuarioResponseDTO toDTO(Usuario usuario) {
        UsuarioResponseDTO dto = new UsuarioResponseDTO();
        dto.setIdentificador(usuario.getId() == null ? null : usuario.getId().toString());
        dto.setUsername(usuario.getUsername());
        dto.setEmail(usuario.getEmail());
        dto.setRoles(usuario.getRoles().stream()
            .map(Rol::getNombre)
            .sorted()
            .toList());
        return dto;
    }
}

```

Comprueba que la ampliación del servicio compila y conserva la suite ya existente. El flujo HTTP se verificará manualmente en los pasos 9 a 11; no adelantamos aquí un test de integración que la fuente todavía no ha introducido:

```
./mvnw -B test
```

Pregunta de reflexión: ¿Por qué el refresh genera también un refresh token nuevo en lugar de conservar indefinidamente el anterior?

Respuesta razonada: Porque la rotación limita la reutilización de una credencial robada. Cuando un refresh se consume, se entrega otro y el anterior deja de ser válido; así el replay puede detectarse y rechazarse.

Paso 7 - Ampliar el AuthController

Endpoints:

```

POST /api/v1/auth/registro
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout

```

Login y refresh reciben DTO validado; logout extrae el Bearer y delega revocación.

Los endpoints del controlador quedan explícitos:

```
@PostMapping("/login")
public ResponseEntity<LoginResponseDTO> login(
    @Valid @RequestBody LoginRequestDTO request) {
    return ResponseEntity.ok(authService.login(request));
}

@PostMapping("/refresh")
public ResponseEntity<LoginResponseDTO> refresh(
    @Valid @RequestBody RefreshRequestDTO request) {
    return ResponseEntity.ok(authService.refresh(request.getRefreshToken()));
}

./mvnw -B -DskipTests compile
```

Pregunta de reflexión: ¿Por qué el endpoint de login debe poder invocarse sin un Bearer token previo?

Respuesta razonada: Porque precisamente es la operación que transforma credenciales válidas en tokens. Si exigiera un token antes de entrar, ningún cliente podría iniciar el proceso de autenticación.

Paso 8 - Configurar la seguridad para permitir login

M6 **no** abre /api/v1/auth/** completo. Sólo son públicos:

```
.requestMatchers(
    HttpMethod.POST,
    "/api/v1/auth/registro",
    "/api/v1/auth/login",
    "/api/v1/auth/refresh")
.permitAll()
```

En la versión final, logout queda autenticado y un endpoint futuro bajo /auth no se hace público accidentalmente.

Login, registro y refresh deben poder invocarse sin credencial previa; logout no.

Tras ajustar los matchers, verifica que no has cerrado accidentalmente el login:

```
./mvnw -B test
```

Pregunta de reflexión: ¿Qué pasa si olvidas permitir /api/v1/auth/login?

Respuesta razonada: La cadena de seguridad rechaza la propia petición de login antes de que el controlador pueda validar las credenciales, de modo que ningún cliente puede obtener su primer token.

Paso 9 - Arrancar y probar el login

```
curl -i -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"ana","password":"ana123"}'
```

La respuesta debe ser 200 e incluir access token, refresh token, tipo Bearer y expiración.

Prueba contraseña incorrecta: **401 CREDENCIALES_INVALIDAS**, no 500 y sin indicar si el username existe.

Una respuesta correcta tiene esta forma:

```
{
  "access_token": "eyJ...",
```

```
"refresh_token": "eyJ...",
"token_type": "Bearer",
"expires_in": 3600
}
```

Pregunta de reflexión: ¿Qué contiene el access_token? ¿Qué contiene el refresh_token?

Respuesta razonada: El access token contiene la identidad y los datos mínimos necesarios para autenticar y autorizar peticiones durante un periodo corto. El refresh token contiene la información mínima para solicitar una renovación y dura más; no debe utilizarse para acceder directamente a recursos.

Paso 10 - Decodificar el token

Usa el decoder local de 6.5 o jwt.io **sólo con tokens didácticos/locales**. El access debe mostrar sub, jti, tipo=access, id, email, roles, iat y exp. Un payload representativo es:

```
{
  "sub": "ana",
  "jti": "550e8400-e29b-41d4-a716-446655440000",
  "tipo": "access",
  "id": 2,
  "email": "ana@educacion.gob.es",
  "roles": ["ROLE_USER"],
  "iat": 1700000000,
  "exp": 1700003600
}
```

Regla: visualizar no equivale a verificar.

Puedes inspeccionar el payload de un token didáctico con el decoder local de 6.5:

```
java -cp target/test-classes es.mecd.demo.miproyecto.jwt.JwtManual "$TOKEN"
```

Pregunta de reflexión: ¿Qué campos del payload son claims registrados y cuáles son personalizados?

Respuesta razonada: sub, jti, iat y exp son claims registrados por JWT. Campos como tipo, id, email y roles son claims privados definidos por nuestra aplicación.

Paso 11 - Probar el refresh

```
curl -i -X POST http://localhost:8080/api/v1/auth/refresh \
-H "Content-Type: application/json" \
-d '{"refresh_token":"<REFRESH>"}
```

Primer uso: 200 + nuevo par. Un refresh inválido o ya consumido devuelve **400 TOKEN_INVALIDO**, de acuerdo con el comportamiento esperado del ejercicio original.

Prueba también un access token en /refresh: debe ser 400 por tipo incorrecto.

La segunda reutilización del mismo refresh debe fallar; así la rotación queda demostrada y no sólo documentada.

Comprueba también el rechazo de un token de tipo incorrecto:

```
curl -i -X POST http://localhost:8080/api/v1/auth/refresh \
-H "Content-Type: application/json" \
-d '{"refresh_token":"<ACCESS_TOKEN>"}
```

Pregunta de reflexión: ¿Qué pasa si el refresh token ha expirado?

Respuesta razonada: Debe rechazarse y el cliente tiene que iniciar de nuevo el flujo de autenticación. En nuestra implementación también se rechaza un refresh ya consumido, para impedir su reutilización.

Paso 12 - Errores comunes del ejercicio

Error	Síntoma	Corrección
clave corta/Base64 inválido credenciales malas producen 500	arranque falla excepción sin traducir	secreto Base64 >=256 bits CredencialesInvalidasException → 401
login devuelve 401 antes de autenticar	matcher cerrado	abrir sólo login/registro/refresh
expires_in enorme	ms enviados como s	dividir por 1000
access usado como refresh	tipo no validado	esRefreshTokenValido
refresh reutilizable	sin registro de uso	consumir jti
claims leídos sin firma	decoder usado para seguridad	parseSignedClaims
secreto en repo prod	configuración insegura	\${JWT_SECRET}

El tratamiento de errores se materializa en tres piezas concretas:

- `CredencialesInvalidasException.java`: representa el rechazo de usuario/contraseña con código estable `CREDENCIALES_INVALIDAS`;
- `TokenInvalidoException.java`: representa JWT expirado, revocado, mal firmado o de tipo incorrecto con código `TOKEN_INVALIDO`;
- `AuthExceptionHandler.java`: traduce credenciales inválidas a HTTP 401 y errores de refresh/token a HTTP 400, conservando el `ErrorResponse` heredado de M5, incluido `traceId`.

Después de revisar los errores habituales, ejecuta de nuevo la suite:

```
./mvnw -B test
```

Paso 13 - Reto resuelto — Endpoint para logout

`TokenRevocationService` guarda `jti` → exp de access tokens revocados. En 6.6 todavía **no existe `JwtAuthenticationFilter`**: ese filtro se crea en 6.7. Por eso este punto necesita un estado temporal y explícito, sin fingir que la revocación ya está integrada en toda la cadena.

Permite temporalmente **sólo** el POST de logout para que el controlador pueda recibir y verificar directamente el Bearer del ejercicio:

```
.requestMatchers(HttpMethod.POST, "/api/v1/auth/logout").permitAll()
```

El controlador extrae el Bearer y delega en `AuthService.logout(...)`; el servicio valida que sea un access token, obtiene su `jti` y registra la revocación:

```
@PostMapping("/logout")
public ResponseEntity<Void> logout(
    @RequestHeader("Authorization") String authHeader) {
    authService.logout(extraerBearer(authHeader));
    return ResponseEntity.noContent().build();
}

@Service
public class TokenRevocationService {
    private final ConcurrentMap<String, Instant> revokedAccess =
        new ConcurrentHashMap<>();

    public void revokeAccess(String jti, Instant expiration) {
        revokedAccess.put(jti, expiration);
    }

    public boolean isAccessRevoked(String jti) {
        return revokedAccess.containsKey(jti);
    }
}
```

```
}  
}
```

Prueba el logout:

```
curl -i -X POST \  
-H "Authorization: Bearer $TOKEN" \  
http://localhost:8080/api/v1/auth/logout
```

Esperado en **6.6**: 204 No Content y el jti queda registrado como revocado.

Todavía no uses la reutilización del token como prueba de revocación efectiva. Sin el filtro Bearer de 6.7, la cadena no consulta aún TokenRevocationService en cada petición protegida. En 6.7 introduciremos JwtAuthenticationFilter, volveremos a proteger logout dentro del flujo Bearer normal y entonces sí comprobaremos que reutilizar el mismo access token produce 401.

Pregunta de reflexión: ¿Por qué registrar la revocación en 6.6 no basta todavía para impedir que el mismo token autentique otra petición?

Respuesta razonada: Porque falta la pieza que consulta esa revocación en cada petición. El servicio puede guardar el jti, pero hasta 6.7 ningún filtro Bearer usa ese dato antes de construir la identidad. La revocación completa aparece cuando JwtAuthenticationFilter valida el token y comprueba además que su jti no esté revocado.

Resultado esperado global

- Dependencias jjwt y clave de firma configuradas de forma externalizable.
- Un JwtService capaz de generar y validar access/refresh tokens.
- Login y refresh que devuelven tokens con expiración y claims controlados.
- Logout capaz de validar el Bearer y registrar la revocación; la aplicación efectiva de esa revocación a peticiones posteriores queda para 6.7.

Resumen técnico del ejercicio

- Módulos jjwt-api, jjwt-impl y jjwt-jackson.
- SecretKey, Jwts.builder() y validación de claims.
- Access token frente a refresh token.
- AuthenticationManager, rotación/revocación y buenas prácticas operacionales.

Conclusión y enlace al siguiente punto

El servidor ya puede emitir y renovar tokens firmados. En 6.7 construiremos la pieza que falta: el filtro que interpreta el Bearer token en cada petición y reconstruye la identidad.

Práctica 6.7 - Filtro JWT

Contexto del ejercicio: vamos a implementar un filtro JWT que intercepte las peticiones, extraiga el token de la cabecera Authorization, lo verifique con JwtService y coloque un UsuarioPrincipal en el SecurityContext. Después registraremos el filtro en la cadena de seguridad y comprobaremos que los endpoints protegidos funcionan con el token.

Requisitos previos: tener el proyecto mi-proyecto con JwtService y UsuarioPrincipal preparados en 6.6 y 6.4.

Paso 1 - Verificar el JwtService y el UsuarioPrincipal

Abre JwtService y comprueba que existe extraerClaims(...). Abre también UsuarioPrincipal y verifica que dispone del constructor con:

```
(Long id, String username, String email,  
Collection<? extends GrantedAuthority> authorities)
```

El filtro necesita leer los claims porque construirá el principal a partir de la información ya firmada dentro del token.

Pregunta de reflexión: ¿Por qué el filtro necesita extraerClaims(...)?

Paso 2 - Crear el JwtAuthenticationFilter

Crea JwtAuthenticationFilter en el paquete auth. La implementación final del snapshot 6.7 es:

```
@Component  
public class JwtAuthenticationFilter extends OncePerRequestFilter {  
  
    private static final Logger LOG =  
        LoggerFactory.getLogger(JwtAuthenticationFilter.class);  
  
    private final JwtService jwtService;  
    private final TokenRevocationService revocation;  
  
    public JwtAuthenticationFilter(  
        JwtService jwtService,  
        TokenRevocationService revocation) {  
        this.jwtService = jwtService;  
        this.revocation = revocation;  
    }  
  
    @Override  
    protected void doFilterInternal(  
        HttpServletRequest request,  
        HttpServletResponse response,  
        FilterChain filterChain)  
        throws ServletException, IOException {  
  
        String authHeader = request.getHeader("Authorization");  
  
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {  
            filterChain.doFilter(request, response);  
            return;  
        }  
  
        String token = authHeader.substring(7);  
  
        if (!jwtService.esAccessTokenValido(token)) {  
            filterChain.doFilter(request, response);  
            return;  
        }  
  
        try {  
            Claims claims = jwtService.extraerClaims(token);  
  
            if (claims.getId() == null  
                || revocation.isAccessRevoked(claims.getId())) {  
                filterChain.doFilter(request, response);  
                return;  
            }  
  
            Number rawId = claims.get("id", Number.class);
```

```

        Long id = rawId == null ? null : rawId.longValue();
        String username = claims.getSubject();
        String email = claims.get("email", String.class);
        List<GrantedAuthority> authorities =
            jwtService.extraerAuthorities(token);

        UsuarioPrincipal principal = new UsuarioPrincipal(
            id, username, email, authorities);

        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(
                principal, null, authorities);

        authentication.setDetails(
            new WebAuthenticationDetailsSource()
                .buildDetails(request));

        SecurityContextHolder.getContext()
            .setAuthentication(authentication);

        LOG.debug(
            "Usuario {} autenticado en {}",
            username,
            request.getRequestURI());
    } catch (RuntimeException ex) {
        SecurityContextHolder.clearContext();
        LOG.debug(
            "JWT no autenticó {}",
            request.getRequestURI());
    }

    filterChain.doFilter(request, response);
}
}

```

La lógica corresponde a la del PDF original: leer Authorization, exigir Bearer, validar el JWT, extraer claims y authorities, construir UsuarioPrincipal, crear un Authentication autenticado y guardarlo en SecurityContextHolder.

Hay tres ajustes de compatibilidad y continuidad con nuestro baseline acumulativo:

- usamos `esAccessTokenValido(...)` en lugar de aceptar cualquier JWT válido, para que un refresh token no pueda autenticar recursos;
- leemos `id` como `Number` y después usamos `longValue()`, porque según el parser un número JSON puede materializarse como `Integer` o `Long`;
- si se implementó el reto opcional de logout de 6.6, el filtro comprueba también si el `jti` está revocado.

Pregunta de reflexión: ¿Qué pasa si el token es válido pero el usuario ya no existe en la base de datos?

Paso 3 - Registrar el filtro en la cadena

Modifica `SecurityConfig` para dejar los endpoints de autenticación y públicos abiertos, exigir ADMIN en la zona administrativa, usar sesiones stateless y registrar el filtro antes de `UsernamePasswordAuthenticationFilter`:

```

@Bean
SecurityFilterChain securityFilterChain(
    HttpSecurity http,

```

```

        JwtAuthenticationFilter jwtFilter) throws Exception {

    http
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.POST,
                "/api/v1/auth/registro",
                "/api/v1/auth/login",
                "/api/v1/auth/refresh").permitAll()
            .requestMatchers(
                "/api/v1/public/**",
                "/h2-console/**",
                "/swagger-ui/**",
                "/swagger-ui.html",
                "/v3/api-docs/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
            .requestMatchers("/api/v1/gestor/**")
                .hasAnyRole("GESTOR", "ADMIN")
            .anyRequest().authenticated())
        .exceptionHandling(ex -> ex
            .authenticationEntryPoint(
                new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED))
            .accessDeniedHandler((request, response, denied) ->
                response.setStatus(HttpStatus.FORBIDDEN.value())))
        .addFilterBefore(
            jwtFilter,
            UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

```

El PDF introduce en 6.7 STATELESS, addFilterBefore(...) y la regla de ADMIN. En el baseline actual añadimos dos handlers **mínimos de estado** sólo para conservar los códigos que el propio ejercicio exige en un servidor real: 401 cuando no hay identidad válida y 403 cuando sí existe identidad pero falta el rol. Los handlers JSON con contrato completo se introducen, como en el original, en 6.8.

Pregunta de reflexión: ¿Qué pasaría si no se configurara STATELESS?

Paso 4 - Desactivar HTTP Basic

A partir de 6.7 ya no se usa HTTP Basic. No debe existir:

```
.httpBasic(Customizer.withDefaults())
```

Usuario y contraseña sólo se envían a /api/v1/auth/login; las peticiones posteriores usan Authorization: Bearer <token>.

Pregunta de reflexión: ¿Qué pasa si mantienes HTTP Basic activo junto con JWT?

Paso 5 - Arrancar y probar

Reinicia la aplicación y obtén un access token para Ana:

```

TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"ana","password":"ana123"}' | jq -r '.access_token')

echo "Token: $TOKEN"

```

Prueba las tres situaciones del ejercicio original:

```
# Sin token a endpoint protegido -> 401
curl -i http://localhost:8080/api/v1/perfil

# Con token -> 200
curl -i -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/v1/perfil

# Endpoint público sin token -> 200
curl -i http://localhost:8080/api/v1/public/info
```

Resultado esperado: **401 / 200 / 200**.

Pregunta de reflexión: ¿Qué diferencia hay entre las tres respuestas?

Paso 6 - Verificar el UsuarioPrincipal

PerfilController recibe ahora directamente el principal construido por el filtro:

```
@GetMapping
public Map<String, Object> perfil(
    @AuthenticationPrincipal UsuarioPrincipal user) {
    return Map.of(
        "id", user.getId(),
        "username", user.getUsername(),
        "email", user.getEmail(),
        "authorities", user.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .toList()
    );
}
```

Prueba:

```
curl -H "Authorization: Bearer $TOKEN" \
  http://localhost:8080/api/v1/perfil
```

Con el estado determinista del inicializador, Ana es el usuario id=2, de modo que verás un resultado equivalente a:

```
{
  "id": 2,
  "username": "ana",
  "email": "ana@educacion.gob.es",
  "authorities": ["ROLE_USER"]
}
```

Pregunta de reflexión: ¿Por qué ahora sí funciona @AuthenticationPrincipal UsuarioPrincipal?

Paso 7 - Probar con rol ADMIN

Primero inicia sesión como admin:

```
TOKEN_ADMIN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123"}' | jq -r '.access_token')
```

```
curl -i -H "Authorization: Bearer $TOKEN_ADMIN" \
  http://localhost:8080/api/v1/admin/usuarios
```

El resultado debe ser **200**.

Ahora usa un token de Ana:

```
TOKEN_ANA=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"ana","password":"anal23"}' | jq -r '.access_token')
```

```
curl -i -H "Authorization: Bearer $TOKEN_ANA" \
http://localhost:8080/api/v1/admin/usuarios
```

El resultado debe ser **403 Forbidden**: Ana está autenticada correctamente, pero no tiene ROLE_ADMIN.

Pregunta de reflexión: ¿Qué diferencia hay entre el 403 de Ana y un 401?

Paso 8 - Probar con un token inválido

```
curl -i -H "Authorization: Bearer tokenInvalido" \
http://localhost:8080/api/v1/perfil
```

Resultado esperado: **401**. El filtro no crea una identidad válida y la regla authenticated() rechaza la petición.

Pregunta de reflexión: ¿Qué diferencia hay entre un token inválido y uno expirado?

Paso 9 - Probar con la cabecera mal formada

El PDF pide probar el token sin el prefijo Bearer:

```
curl -i -H "Authorization: $TOKEN_ANA" \
http://localhost:8080/api/v1/perfil
```

Resultado esperado: **401**. El filtro ignora la cabecera porque no empieza por Bearer.

Pregunta de reflexión: ¿Por qué el filtro es estricto con el prefijo Bearer?

Paso 10 - Depurar el filtro

Activa temporalmente:

```
logging.level.org.springframework.security=DEBUG
logging.level.es.mecd.demo.miproyecto.auth=DEBUG
```

Con una petición válida podrás observar que la cadena protege la ruta y que el filtro coloca al usuario en el SecurityContext.

No registres access tokens, refresh tokens ni secretos completos.

Pregunta de reflexión: ¿Qué información aportan los logs que no tenías antes?

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
401 siempre	el filtro no se ejecuta o no autentica	revisar addFilterBefore
403 con token válido	authorities vacías o incorrectas	construir Authentication con authorities
la petición queda bloqueada	falta filterChain.doFilter(...)	continuar siempre la cadena
NullPointerException	se usa authHeader sin comprobarlo	comprobar null antes
500 al parsear token	excepción JWT no controlada	validar/capturar la excepción

Error	Causa	Solución
token no leído aparece sesión	cabecera sin Bearer falta STATELESS	verificar formato fijar SessionCreationPolicy.STATELESS
endpoint público devuelve 401	filtro no tolera ausencia de token	dejar pasar sin autenticar

Paso 12 - Reto resuelto: expresión SpEL con el ID del usuario

Añade al PerfilController:

```
@GetMapping("/{id}")
@PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
public Map<String, Object> perfilPorId(
    @PathVariable Long id,
    @AuthenticationPrincipal UsuarioPrincipal user) {
    return Map.of(
        "id", user.getId(),
        "username", user.getUsername(),
        "email", user.getEmail()
    );
}
```

La expresión compara el parámetro id con authentication.principal.id. Un administrador puede acceder a cualquier ID gracias a or hasRole('ADMIN').

Con el inicializador del curso, admin se crea primero (id=1) y Ana después (id=2). Prueba exactamente los tres casos del PDF:

```
# Ana consulta su propio perfil (id=2) -> 200
curl -i -H "Authorization: Bearer $TOKEN_ANA" \
    http://localhost:8080/api/v1/perfil/2
```

```
# Ana consulta el perfil de admin (id=1) -> 403
curl -i -H "Authorization: Bearer $TOKEN_ANA" \
    http://localhost:8080/api/v1/perfil/1
```

```
# Admin consulta cualquier perfil -> 200
curl -i -H "Authorization: Bearer $TOKEN_ADMIN" \
    http://localhost:8080/api/v1/perfil/2
```

Pregunta de reflexión: ¿Qué ocurriría si el principal fuera un String en lugar de un UsuarioPrincipal?

Resultado esperado global

Al finalizar 6.7 debes tener:

- un JwtAuthenticationFilter que extrae y verifica el token;
- el filtro registrado en la cadena;
- política STATELESS;
- HTTP Basic desactivado;
- PerfilController usando @AuthenticationPrincipal UsuarioPrincipal;
- endpoint con expresión SpEL sobre authentication.principal.id;
- endpoints protegidos funcionando con JWT;
- endpoints públicos funcionando sin token.

Resumen técnico del ejercicio

El punto 6.7 conecta JWT con Spring Security mediante OncePerRequestFilter, SecurityContextHolder, UsernamePasswordAuthenticationToken, addFilterBefore(...) y una política stateless.

Conclusión y enlace al siguiente punto

El filtro JWT es la pieza que convierte un token firmado en una identidad que Spring Security puede autorizar. En 6.8 añadiremos los manejadores JSON específicos para 401/403, el rol GESTOR y el flujo completo de protección de endpoints.

Práctica 6.8 - Protección de endpoints con JWT

Contexto del ejercicio: Vamos a integrar todo lo construido en un SecurityFilterChain completo, con manejadores 401/403 personalizados, reglas de autorización y el flujo completo registro → login → petición → refresh → logout.

Requisitos previos: Tener completado el filtro JWT del punto 6.7.

Paso 1 - Crear el JwtAuthenticationEntryPoint

Implementa AuthenticationEntryPoint y delega en SecurityErrorWriter.

La respuesta 401 debe ser JSON ErrorResponse con timestamp, status, codigo=NO_AUTENTICADO, mensaje, path y traceId.

Por qué no @RestControllerAdvice: la petición puede fallar dentro de Security antes de entrar en MVC.

```
package es.mecd.demo.miproyecto.auth;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

/** Respuesta JSON uniforme cuando falta autenticación válida. */
@Component
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint {
    private final SecurityErrorWriter errorWriter;

    public JwtAuthenticationEntryPoint(SecurityErrorWriter errorWriter) {
        this.errorWriter = errorWriter;
    }

    @Override
    public void commence(
        HttpServletRequest request,
        HttpServletResponse response,
        AuthenticationException authException) throws IOException {
        errorWriter.write(
            request,
            response,
            HttpServletResponse.SC_UNAUTHORIZED,
            "NO_AUTENTICADO",
            "Se requiere autenticación para acceder a este recurso");
    }
}
```

```
}
}
```

Prueba el manejador sin token:

```
curl -i http://localhost:8080/api/v1/perfil
```

Pregunta de reflexión: ¿Por qué este error no se resuelve únicamente con `@RestControllerAdvice`?

Respuesta razonada: Porque el rechazo puede ocurrir dentro de la cadena de Spring Security antes de que Spring MVC invoque un controlador. `AuthenticationEntryPoint` es la frontera adecuada para traducir ese 401.

Paso 2 - Crear el `JwtAccessDeniedHandler`

Implementa `AccessDeniedHandler` y devuelve el mismo contrato con status 403 y código=ACCESO_DENEGADO.

Los errores 401 y 403 deben utilizar el mismo contrato JSON que el resto de la API, no páginas HTML ni formatos alternativos.

```
package es.mecd.demo.miproyecto.auth;
```

```
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.stereotype.Component;
```

```
/** Respuesta JSON uniforme para un usuario autenticado sin permisos. */
```

```
@Component
```

```
public class JwtAccessDeniedHandler implements AccessDeniedHandler {
    private final SecurityErrorWriter errorWriter;

    public JwtAccessDeniedHandler(SecurityErrorWriter errorWriter) {
        this.errorWriter = errorWriter;
    }
}
```

```
@Override
```

```
public void handle(
    HttpServletRequest request,
    HttpServletResponse response,
    AccessDeniedException accessDeniedException) throws IOException {
    errorWriter.write(
        request,
        response,
        HttpServletResponse.SC_FORBIDDEN,
        "ACCESO_DENEGADO",
        "No tiene permisos para acceder a este recurso");
}
```

```
}
```

Comprueba un usuario autenticado sin permisos:

```
curl -i -H "Authorization: Bearer $USER_TOKEN" \
http://localhost:8080/api/v1/admin/usuarios
```

Pregunta de reflexión: ¿Cuál es la diferencia exacta entre el 401 del paso anterior y este 403?

Respuesta razonada: En 401 no hay una identidad válida para una ruta protegida. En 403 sí la hay, pero sus roles/authorities no permiten ejecutar la operación solicitada.

Paso 3 - Actualizar el SecurityConfig

Cadena final:

```
.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
.authorizeHttpRequests(...)
.exceptionHandling(ex -> ex
    .authenticationEntryPoint(entryPoint)
    .accessDeniedHandler(deniedHandler))
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
```

Reglas públicas específicas, ADMIN, GESTOR/ADMIN y anyRequest().authenticated() al final.

La configuración debe conservar además CORS y OpenAPI se conservan explícitamente.

La configuración completa debe conservar las piezas heredadas y ordenar las reglas específicas antes de la general:

@Bean

```
SecurityFilterChain securityFilterChain(
    HttpSecurity http,
    JwtAuthenticationFilter jwtFilter,
    JwtAuthenticationEntryPoint entryPoint,
    JwtAccessDeniedHandler deniedHandler) throws Exception {
    http
        .csrf(AbstractHttpConfigurer::disable)
        .cors(Customizer.withDefaults())
        .sessionManagement(s -> s
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.POST,
                "/api/v1/auth/registro",
                "/api/v1/auth/login",
                "/api/v1/auth/refresh")
            .permitAll()
            .requestMatchers("/api/v1/public/**").permitAll()
            .requestMatchers("/h2-console/**").permitAll()
            .requestMatchers("/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
            .requestMatchers("/api/v1/gestor/**")
                .hasAnyRole("GESTOR", "ADMIN")
            .anyRequest().authenticated())
        .exceptionHandling(ex -> ex
            .authenticationEntryPoint(entryPoint)
            .accessDeniedHandler(deniedHandler))
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}

./mvnw -B test
```

Pregunta de reflexión: ¿Qué pasaría si anyRequest().authenticated() se declarase antes que los matchers públicos y por rol?

Respuesta razonada: La regla general absorbería peticiones que deberían evaluarse con reglas más concretas o la DSL impediría seguir añadiendo matchers. Las excepciones y roles deben expresarse

antes del caso general.

Paso 4 - Añadir un endpoint de gestor

GestorController expone:

GET /api/v1/gestor/documentos

Y SecurityConfig exige GESTOR o ADMIN. El endpoint existe desde el reto 6.2 y aquí se verifica bajo JWT, no se duplica.

El controlador del gestor es deliberadamente sencillo; la seguridad está en la cadena:

```
@RestController
@RequestMapping("/api/v1/gestor")
public class GestorController {
    @GetMapping("/documentos")
    public Map<String, String> documentos() {
        return Map.of("mensaje", "Lista de documentos del gestor");
    }
}
```

```
curl -i -H "Authorization: Bearer $GESTOR_TOKEN" \
http://localhost:8080/api/v1/gestor/documentos
```

Pregunta de reflexión: ¿Quién puede acceder a /api/v1/gestor/documentos con la regla final?

Respuesta razonada: Una identidad con ROLE_GESTOR o ROLE_ADMIN. Un USER autenticado queda fuera y recibe 403.

Paso 5 - Añadir el rol GESTOR al inicializador

UsuariosInicialesConfig ya es idempotente y conserva:

gestor / gestor123 → USER + GESTOR

Continuidad: migrar a DB/JWT no puede destruir un reto de puntos anteriores.

El inicializador debe materializar el rol y asignarlo al usuario de prueba:

```
Rol rolGestor = obtenerOCrearRol(
    rolRepository, "GESTOR", "Gestor");
Usuario gestor = obtenerOCrearUsuario(
    usuarioRepository, "gestor", "gestor@educacion.gob.es");
prepararUsuario(
    gestor, "gestor123", passwordEncoder, rolUser, rolGestor);
usuarioRepository.save(gestor);
```

Pregunta de reflexión: ¿Qué roles tiene ahora el usuario gestor?

Respuesta razonada: Tiene GESTOR y USER: GESTOR le permite acceder a la zona de gestión y USER conserva las capacidades ordinarias definidas para un usuario del sistema.

Paso 6 - Probar el flujo completo

Gestor:

1. login → token;
2. perfil → 200;
3. /gestor/documentos → 200;
4. /admin/usuarios → 403.

Esto distingue identidad válida de permiso insuficiente.

Ejecuta el flujo completo con el mismo usuario:

```
GESTOR_TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"gestor","password":"gestor123"}' | jq -r '.access_token')

curl -i -H "Authorization: Bearer $GESTOR_TOKEN" http://localhost:8080/api/v1/perfil
curl -i -H "Authorization: Bearer $GESTOR_TOKEN" http://localhost:8080/api/v1/gestor/documentos
curl -i -H "Authorization: Bearer $GESTOR_TOKEN" http://localhost:8080/api/v1/admin/usuarios
```

Los resultados deben ser 200, 200 y 403.

Pregunta de reflexión: ¿Qué diferencia hay entre el 403 de gestor y un 401?

Respuesta razonada: El 403 demuestra que el token se ha validado y que la identidad está autenticada, pero no posee el rol ADMIN. Un 401 indicaría que no hay una autenticación válida, por ejemplo porque falta el token o no puede verificarse.

Paso 7 - Probar sin token

```
curl -i http://localhost:8080/api/v1/perfil
curl -i http://localhost:8080/api/v1/public/info
```

Esperado: 401 JSON con traceId / 200.

Pregunta de reflexión: ¿Qué formato tiene la respuesta 401? ¿Es consistente con el resto de la API?

Respuesta razonada: Debe ser JSON y respetar el contrato de errores del proyecto: status 401, código estable, mensaje, path, timestamp y traceId. Así la seguridad no introduce un formato distinto al utilizado por el resto de la API.

Paso 8 - Probar el refresh token

Obtén refresh, úsalo una vez y comprueba que devuelve un par nuevo. Reutiliza el antiguo: **400 TOKEN_INVALIDO**, que es el contrato aplicado por AuthExceptionHandler a los errores de refresh.

En este proyecto, la rotación está implementada, no se limita a emitir otro token mientras el anterior sigue válido.

Renueva el par y después intenta reutilizar el refresh anterior:

```
curl -i -X POST http://localhost:8080/api/v1/auth/refresh \
-H "Content-Type: application/json" \
-d '{"refresh_token":"'${REFRESH}'"}'
```

La primera llamada es válida; la reutilización del mismo refresh debe ser rechazada.

Pregunta de reflexión: ¿Qué pasa si intentas usar el mismo refresh token dos veces?

Respuesta razonada: El primer uso consume el refresh y devuelve un nuevo par. El segundo intento debe rechazarse porque ese jti ya fue consumido; así se limita el replay de una credencial robada.

Paso 9 - Probar el logout

Login → perfil 200 → logout 204 → mismo access token en perfil → 401.

El filtro consulta jti revocado antes de construir identidad.

Prueba la revocación de access de extremo a extremo:

```
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
curl -i -X POST -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/auth/logout
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8080/api/v1/perfil
```

La secuencia debe ser 200 → 204 → 401.

Pregunta de reflexión: ¿Qué limitación tiene la lista negra en memoria?

Respuesta razonada: Sólo protege la instancia que guarda esa lista y se pierde al reiniciar. En producción distribuida se necesita una fuente compartida de revocación o una estrategia equivalente.

Paso 10 - Escribir tests de integración

La fuente introduce aquí un **test de la capa web del AuthController**. Se conserva ese aprendizaje en AuthControllerTest y se moderniza @MockBean a @MockitoBean para Spring Boot 3.5.

Como JwtAuthenticationFilter ya existe desde 6.7, el slice debe proporcionar también sus colaboradores para que Spring pueda construir el contexto. addFilters = false evita ejecutar la cadena durante estas dos pruebas, pero no elimina los beans de filtro del contexto.

```
package es.mecd.demo.miproyecto.auth;
```

```
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
```

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
```

```
@WebMvcTest(AuthController.class)
```

```
@AutoConfigureMockMvc(addFilters = false)
```

```
class AuthControllerTest {
```

```
    @Autowired
```

```
    private MockMvc mockMvc;
```

```
    @MockitoBean
```

```
    private AuthService authService;
```

```
    @MockitoBean
```

```
    private JwtService jwtService;
```

```
    @MockitoBean
```

```
    private TokenRevocationService tokenRevocationService;
```

```
    @Test
```

```
    void login_debeDevolver200_cuandoCredencialesValidas() throws Exception {
        LoginResponseDTO response = new LoginResponseDTO("token", "refresh", 3600);
        when(authService.login(any())).thenReturn(response);
```

```
        mockMvc.perform(post("/api/v1/auth/login")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("""
                                {"username": "ana", "password": "ana123"}
                                """))
                .andExpect(status().isOk());
    }
```

```

        .andExpect(jsonPath("$.access_token").value("token"))
        .andExpect(jsonPath("$.token_type").value("Bearer"));
    }

    @Test
    void login_debeDevolver401_cuandoCredencialesInvalidas() throws Exception {
        when(authService.login(any()))
            .thenThrow(new CredencialesInvalidasException());

        mockMvc.perform(post("/api/v1/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"username":"ana","password":"incorrecta"}
                """))
            .andExpect(status().isUnauthorized())
            .andExpect(jsonPath("$.codigo")
                .value("CREDENCIALES_INVALIDAS"));
    }
}

```

La segunda prueba moderniza únicamente el contrato de error: el ejemplo histórico esperaba el 409 genérico de `NegocioException`, mientras que M6 ya dispone de `CredencialesInvalidasException` y devuelve 401 `CREDENCIALES_INVALIDAS`.

Ejecuta primero este test y después la suite acumulada:

```

./mvnw -B -Dtest=AuthControllerTest test
./mvnw -B test

```

Pregunta de reflexión: ¿Qué cubren los dos tests?

Respuesta razonada: El primero verifica que el controlador devuelve el par de tokens y Bearer cuando el servicio acepta las credenciales; el segundo comprueba que una autenticación rechazada se transforma en el contrato HTTP 401 estable de M6. El mecanismo JWT completo y su filtro se probarán de extremo a extremo en 6.9.

Paso 11 - Errores comunes del ejercicio

Error	Causa	Solución
401 HTML	entrypoint ausente	configurar handler JSON
403 no uniforme	handler ausente	<code>JwtAccessDeniedHandler</code>
gestor 403	rol no emitido/asignado	revisar initializer/claims
refresh viejo funciona dos veces	sin consumo de jti	rotación real
logout no invalida	filtro no consulta revocación	blacklist por jti
CORS falla tras Security	<code>.cors()</code> omitido	integrar CORS M5
OpenAPI 401	las rutas públicas dejaron de estar permitidas	rutas Swagger/docs públicas
<code>anyRequest</code> tapa públicos	orden incorrecto	regla general al final

Tras corregir un problema de configuración, vuelve a ejecutar toda la suite:

```

./mvnw -B test

```

Paso 12 - Reto resuelto — Endpoint para consultar el propio usuario

GET `/api/v1/perfil/usuario`

Devuelve id, username y email desde `UsuarioPrincipal`, sin acceso a repositorio.

```
curl -H "Authorization: Bearer $TOKEN" \  
http://localhost:8080/api/v1/perfil/usuario
```

Los datos devueltos deben coincidir con los claims ya verificados del principal.

El endpoint usa directamente el principal ya construido por el filtro:

```
@GetMapping("/usuario")  
public Map<String, Object> usuario(  
    @AuthenticationPrincipal UsuarioPrincipal user) {  
    return Map.of(  
        "id", user.getId(),  
        "username", user.getUsername(),  
        "email", user.getEmail());  
}
```

Pregunta de reflexión: ¿Por qué es mejor usar los claims del token que consultar la base de datos?

Respuesta razonada: Porque esos claims ya han sido verificados criptográficamente por el filtro y contienen los datos de identidad que este endpoint necesita. Evitar una consulta extra mantiene el flujo stateless y reduce latencia; la base de datos sigue siendo necesaria cuando la operación requiere estado actual que no viaja en el token.

Resultado esperado global

- Handlers JSON específicos para 401 y 403.
- SecurityFilterChain completo con stateless, filtro JWT y reglas ordenadas.
- Rol y endpoint GESTOR integrados.
- Flujo completo login → petición → refresh → logout probado.

Resumen técnico del ejercicio

- Configuración JWT completa, CSRF off y sesiones stateless.
- AuthenticationEntryPoint 401 y AccessDeniedHandler 403 con JSON.
- Allowlist explícita, reglas por rol y orden de matchers.
- Pruebas de flujo completo y errores coherentes con M5.

Conclusión y enlace al siguiente punto

La arquitectura JWT queda integrada de extremo a extremo. El último punto, 6.9, convierte ese contrato de seguridad en una suite de pruebas reproducible que detecte regresiones.

Práctica 6.9 - Testing de seguridad

Contexto del ejercicio: Vamos a escribir tests de seguridad para UsuarioController, AuthController, el endpoint de gestor y el filtro JWT. Verificaremos autenticación, autorización y estructura de errores.

Requisitos previos: Tener el proyecto con la configuración de seguridad completa del punto 6.8.

Paso 1 - Añadir la dependencia spring-security-test

El PDF original introduce formalmente aquí spring-security-test. Añade la dependencia con scope=test. En el snapshot de referencia puede aparecer ya heredada de 6.4 porque allí se ejecutan tests de autorización que necesitan SecurityMockMvcRequestPostProcessors.user(...); en 6.9 se conserva y se amplía su uso con el resto de herramientas de Spring Security Test.

```
<dependency>  
<groupId>org.springframework.security</groupId>
```

```
<artifactId>spring-security-test</artifactId>
<scope>test</scope>
</dependency>
```

Deben estar disponibles `@WithMockUser`, `@WithUserDetails` y los post-processors de Spring Security Test.

Recarga y verifica que las clases de test están disponibles:

```
./mvnw -B -DskipTests test-compile
```

Pregunta de reflexión: ¿Por qué spring-security-test tiene scope=test?

Respuesta razonada: Porque sus anotaciones, post-processors y matchers se utilizan únicamente para construir escenarios de prueba. No deben formar parte del classpath de ejecución de la aplicación desplegada.

Paso 2 - Crear el test del UsuarioController

`UsuarioControllerSecurityTest` usa contexto completo de test para validar la configuración real y cubre:

- anónimo → 401 + NO_AUTENTICADO;
- USER → 403 + ACCESO_DENEGADO;
- ADMIN → 200.

M6 evita `@MockBean`. Cuando se necesita sustituir un bean Spring, el contrato exige `@MockitoBean`.

El ejemplo del PDF usa `@MockBean`. En Spring Boot 3.5 modernizamos únicamente esa pieza a `@MockitoBean`, manteniendo los tres escenarios originales:

```
@WebMvcTest(UsuarioController.class)
class UsuarioControllerSecurityTest {
    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private UsuarioService usuarioService;

    @Test
    void listar_debeDevolver401_cuandoNoAutenticado() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(roles = "USER")
    void listar_debeDevolver403_cuandoRolUser() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void listar_debeDevolver200_cuandoRolAdmin() throws Exception {
        when(usuarioService.listarTodos()).thenReturn(List.of());
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isOk());
    }
}
```

```
./mvnw -B -Dtest=UsuarioControllerSecurityTest test
```

Pregunta de reflexión: ¿Por qué hay que probar 401, 403 y 200 por separado?

Respuesta razonada: Porque representan tres contratos distintos: ausencia de identidad, identidad sin permiso e identidad autorizada. Probar sólo el 200 no detectaría una apertura accidental de la ruta.

Paso 3 - Ejecutar los tests

Ejecutamos la suite **acumulativa completa**, no sólo los tests nuevos:

```
./mvnw -B clean verify
```

El objetivo final es **58 tests**: los 32 heredados de M5 más 26 tests M6, sin fallos, errores ni skipped, y con Checkstyle a cero.

Cinco `@WebMvcTest` heredados se mantienen como tests de controlador/error/CORS, no como tests de seguridad. Al entrar Spring Security en el classpath de M6 se evolucionan con `@AutoConfigureMockMvc(addFilters = false)` para que sigan verificando su contrato M5 y, además, con `@MockitoBean` para los dos colaboradores del `JwtAuthenticationFilter` (`JwtService` y `TokenRevocationService`). Esto último es necesario porque desactivar la aplicación de filtros en `MockMvc` **no desactiva la creación del bean filtro** durante el arranque del slice. La seguridad se prueba por separado en los tests dedicados de este módulo y en la prueba de ejecución real.

Los cinco ficheros heredados que reciben esa modificación mínima y exclusivamente de aislamiento del slice son:

- `AlumnoControllerTest.java`;
- `ErrorHandlingTest.java`;
- `CorsConfigTest.java`;
- `ExpedienteControllerTest.java`;
- `FicheroExceptionHandlerTest.java`.

En cada uno de esos cinco slices heredados, el patrón de aislamiento queda explícito:

```
@AutoConfigureMockMvc(addFilters = false)
```

```
@MockitoBean
private JwtService jwtService;
```

```
@MockitoBean
private TokenRevocationService tokenRevocationService;
```

Si cambia una regla y abre `/admin`, los escenarios `USER/anónimo` de la suite M6 deben fallar inmediatamente.

Paso 4 - Testear el endpoint de gestor

El ejemplo de la fuente comprueba explícitamente los tres roles del contrato de `/api/v1/gestor/documentos`:

```
@Test
@WithMockUser(roles = "GESTOR")
void gestor_debeDevolver200_cuandoRolGestor() throws Exception {
    mockMvc.perform(get("/api/v1/gestor/documentos"))
        .andExpect(status().isOk());
}

@Test
@WithMockUser(roles = "ADMIN")
void gestor_debeDevolver200_cuandoRolAdmin() throws Exception {
    mockMvc.perform(get("/api/v1/gestor/documentos"))
        .andExpect(status().isOk());
}
```

```

@Test
@WithMockUser(roles = "USER")
void gestor_debeDevolver403_cuandoRolUser() throws Exception {
    mockMvc.perform(get("/api/v1/gestor/documentos"))
        .andExpect(status().isForbidden());
}

```

En el proyecto final estos casos viven en GestorSecurityTest y se ejecutan contra la configuración real.

```
./mvnw -B -Dtest=GestorSecurityTest test
```

Pregunta de reflexión: ¿Por qué el usuario USER debe recibir 403 y no 401 al intentar acceder al endpoint de gestor?

Respuesta razonada: Porque USER ya está autenticado. Lo que falla es la autorización, ya que no posee GESTOR ni ADMIN; por eso corresponde 403 Forbidden.

Paso 5 - Testear el endpoint público

La fuente crea PublicControllerSecurityTest para impedir que una futura modificación convierta accidentalmente una ruta pública en privada. Conservamos el test completo y lo ejecutamos contra el contexto final:

```

package es.mecd.demo.miproyecto.common.controller;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class PublicControllerSecurityTest {
    @Autowired
    private MockMvc mockMvc;

    @Test
    void info_debeDevolver200_sinAutenticacion() throws Exception {
        mockMvc.perform(get("/api/v1/public/info"))
            .andExpect(status().isOk());
    }
}

./mvnw -B -Dtest=PublicControllerSecurityTest test

```

Pregunta de reflexión: ¿Qué pasaría si el endpoint público requiriera autenticación por error? ¿El test lo detectaría?

Respuesta razonada: Sí. La petición no aporta identidad; si la ruta dejara de estar en la allowlist, pasaría de 200 a 401 y el test fallaría.

Paso 6 - Testear el login

El PDF pide crear AuthControllerSecurityTest. En la versión corregida lo convertimos en un **test de integración real**, porque un test con @WebMvcTest y addFilters=false sólo demostraría el contrato del controlador y podría pasar aunque la seguridad estuviera rota.

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class AuthControllerSecurityTest {
    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @Test
    void login_debeDevolver200ConTokens_cuandoCredencialesValidas()
        throws Exception {
        mockMvc.perform(post("/api/v1/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"username\":\"ana\",\"password\":\"ana123\"}"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.access_token").isNotEmpty())
            .andExpect(jsonPath("$.refresh_token").isNotEmpty())
            .andExpect(jsonPath("$.token_type").value("Bearer"))
            .andExpect(jsonPath("$.expires_in").value(3600));
    }

    @Test
    void login_debeDevolver401_cuandoCredencialesInvalidas()
        throws Exception {
        mockMvc.perform(post("/api/v1/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"username\":\"ana\",\"password\":\"incorrecta\"}"))
            .andExpect(status().isUnauthorized())
            .andExpect(jsonPath("$.codigo").value("CREDENCIALES_INVALIDAS"));
    }
}
```

Aquí no hay AuthService simulado y los filtros están activos. Por tanto el test verifica el AuthenticationManager, el usuario de base de datos, el encoder, la generación JWT y el contrato HTTP de login.

Paso 7 - Testear el refresh

El refresh se prueba con un **refresh token real obtenido del login**, no con la cadena ficticia refreshValido:

```
@Test
void refresh_debeEmitirUnNuevoParDeTokens() throws Exception {
    JsonNode login = loginAna();
    String refresh = login.get("refresh_token").asText();

    mockMvc.perform(post("/api/v1/auth/refresh")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"refresh_token\":\"" + refresh + "\"}"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.access_token").isNotEmpty());
}
```

```
        .andExpect(jsonPath("$.refresh_token").isEmpty());
    }
}
```

Con esto comprobamos de extremo a extremo firma, expiración, tipo=refresh, usuario persistido y rotación/consumo del refresh token.

Paso 8 - Testear el logout

El logout debe demostrar **revocación efectiva**, no limitarse a comprobar que el controlador devuelve 204:

```
@Test
void logout_debeRevocarElAccessToken() throws Exception {
    JsonNode login = loginAna();
    String access = login.get("access_token").asText();

    mockMvc.perform(get("/api/v1/perfil")
        .header("Authorization", "Bearer " + access))
        .andExpect(status().isOk());

    mockMvc.perform(post("/api/v1/auth/logout")
        .header("Authorization", "Bearer " + access))
        .andExpect(status().isNoContent());

    mockMvc.perform(get("/api/v1/perfil")
        .header("Authorization", "Bearer " + access))
        .andExpect(status().isUnauthorized())
        .andExpect(jsonPath("$.codigo").value("NO_AUTENTICADO"));
}
```

La secuencia obligatoria es **200 → 204 → 401**. Ésta es la evidencia de que TokenRevocationService está conectado al filtro JWT y no es simplemente una clase que almacena datos sin afectar a la autenticación.

Paso 9 - Testear con post-processors

La fuente exige demostrar SecurityMockMvcRequestPostProcessors.user(...) con las dos identidades concretas. Conservamos ambos ejemplos:

```
import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.user;
```

```
@Test
void listar_debeDevolver200_cuandoRolAdminConPostProcessor()
    throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios")
        .with(user("admin").roles("ADMIN")))
        .andExpect(status().isOk());
}
```

```
@Test
void listar_debeDevolver403_cuandoRolUserConPostProcessor()
    throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios")
        .with(user("ana").roles("USER")))
        .andExpect(status().isForbidden());
}
```

.with(user(...)) aplica una identidad sólo a esa petición.

```
./mvnw -B -Dtest=UsuarioControllerSecurityTest test
```

Pregunta de reflexión: ¿Qué ventaja tiene usar `user()` en lugar de `@WithMockUser`?

Respuesta razonada: Permite cambiar la identidad por petición de forma explícita, incluso dentro de un mismo método; `@WithMockUser` configura el contexto del test completo.

Paso 10 - Testear con `@WithUserDetails`

La fuente crea deliberadamente un usuario **test** en `@BeforeEach`, le asigna ADMIN y después utiliza `@WithUserDetails("test")`. No sustituimos este ejemplo por el `admin` ya sembrado, porque aquí se quiere demostrar que la anotación carga una identidad real mediante el `UserDetailsService` configurado.

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class UsuarioControllerIntegrationTest {
    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private UsuarioRepository usuarioRepository;

    @Autowired
    private RolRepository rolRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @BeforeEach
    void setUp() {
        if (usuarioRepository.findByUsername("test").isEmpty()) {
            Rol rol = rolRepository.findByNombre("ADMIN")
                .orElseGet(() -> rolRepository.save(
                    new Rol("ADMIN", "Admin")));
            Usuario usuario = new Usuario();
            usuario.setUsername("test");
            usuario.setPassword(passwordEncoder.encode("test123"));
            usuario.setEmail("test@educacion.gob.es");
            usuario.setActivo(true);
            usuario.getRoles().add(rol);
            usuarioRepository.save(usuario);
        }
    }

    @Test
    @WithUserDetails("test")
    void listar_debeDevolver200_conUsuarioDeBD() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isOk());
    }
}

./mvnw -B -Dtest=UsuarioControllerIntegrationTest test
```

Pregunta de reflexión: ¿Qué ventaja tiene `@WithUserDetails` frente a `@WithMockUser` en este caso?

Respuesta razonada: `@WithUserDetails` llama al `UserDetailsService` real y comprueba también la transformación de la identidad persistida y sus roles al principal de Spring Security. `@WithMockUser` no detectaría un fallo en esa carga.

Paso 11 - Testear el filtro JWT

JwtFilterIntegrationTest genera un access token real con JwtService:

- token válido → perfil 200;
- token inválido → 401;
- sin token → 401.

Por qué no jwt() como sustituto: nuestro sistema no es el resource-server OAuth2 de Spring; necesitamos demostrar que **nuestro filtro** procesa una cabecera firmada por **nuestro JwtService**.

`package es.mecd.demo.miproyecto.auth;`

```
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
```

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.web.servlet.MockMvc;
```

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class JwtFilterIntegrationTest {
    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private JwtService jwtService;

    @Autowired
    private UsuarioRepository usuarioRepository;

    @Test
    void perfil_debeDevolver200_conTokenValido() throws Exception {
        Usuario ana = usuarioRepository.findByUsername("ana").orElseThrow();
        String token = jwtService.generarToken(ana);

        mockMvc.perform(get("/api/v1/perfil")
            .header("Authorization", "Bearer " + token))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.username").value("ana"));
    }

    @Test
    void perfil_debeDevolver401_conTokenInvalido() throws Exception {
        mockMvc.perform(get("/api/v1/perfil")
            .header("Authorization", "Bearer tokenInvalido"))
            .andExpect(status().isUnauthorized())
            .andExpect(jsonPath("$.codigo").value("NO_AUTENTICADO"));
    }

    @Test
    void perfil_debeDevolver401_sinToken() throws Exception {
        mockMvc.perform(get("/api/v1/perfil"))
    }
}
```

```

        .andExpect(status().isUnauthorized());
    }
}

```

Ejecuta específicamente la prueba del filtro:

```
./mvnw -B -Dtest=JwtFilterIntegrationTest test
```

Pregunta de reflexión: ¿Por qué esta prueba usa un token real generado por JwtService en vez de sustituir todo por un post-processor jwt()?

Respuesta razonada: Porque el objetivo es probar nuestro propio filtro y nuestro propio parser JWT. Un JWT simulado por la infraestructura OAuth2 de Spring podría demostrar autorización, pero no demostraría que JwtAuthenticationFilter procesa correctamente una cabecera firmada por nuestra aplicación.

Paso 12 - Errores comunes del ejercicio

Error	Causa	Corrección
clases de security test no existen	falta dependencia	spring-security-test
esperaba 403, recibe 401	no hay identidad	preparar usuario antes de roles
esperaba 200, recibe 403	rol incorrecto	revisar ROLE_/matcher
error JSON no coincide	handlers no activos	verificar cadena completa
@WithUserDetails falla	servicio/usuario ausente	@SpringBootTest + datos
filtro no se prueba	se simuló SecurityContext	enviar JWT real en integración
tests lentos innecesarios	todo es @SpringBootTest	usar slice cuando no se necesite mecanismo real
tests dependen de orden	datos compartidos mutables	escenarios independientes

Como control de modernización, comprueba que no quedan usos nuevos de la API obsoleta en los tests M6:

```
grep -R "@MockBean" src/test || true
```

Paso 13 - Reto resuelto — Test completo de autorización

El reto final conserva los cinco escenarios de la fuente y se ejecuta contra la configuración final real:

```

@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class UsuarioControllerCompleteSecurityTest {
    @Autowired
    private MockMvc mockMvc;

    @Test
    void listar_debeDevolver401_cuandoNoAutenticado() throws Exception {
        mockMvc.perform(get("/api/v1/admin/usuarios"))
            .andExpect(status().isUnauthorized())
            .andExpect(jsonPath("$.codigo").value("NO_AUTENTICADO"))
            .andExpect(jsonPath("$.mensaje").exists())
            .andExpect(jsonPath("$.path").value("/api/v1/admin/usuarios"));
    }

    @Test
    @WithMockUser(roles = "USER")

```

```

void listar_debeDevolver403_cuandoRolUser() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isForbidden())
        .andExpect(jsonPath("$.codigo").value("ACCESO_DENEGADO"));
}

@Test
@WithMockUser(roles = "GESTOR")
void listar_debeDevolver403_cuandoRolGestor() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isForbidden())
        .andExpect(jsonPath("$.codigo").value("ACCESO_DENEGADO"));
}

@Test
@WithMockUser(roles = "ADMIN")
void listar_debeDevolver200_cuandoRolAdmin() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isOk());
}

@Test
@WithMockUser(roles = "ADMIN")
void listar_debeDevolverUsuarios_cuandoRolAdmin() throws Exception {
    mockMvc.perform(get("/api/v1/admin/usuarios"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$[0].username").exists())
        .andExpect(jsonPath("$[0].email").exists());
}
}

./mvnw -B -Dtest=UsuarioControllerCompleteSecurityTest test

```

Pregunta de reflexión: ¿Por qué el escenario GESTOR espera 403 y no 401?

Respuesta razonada: Porque GESTOR es una identidad válida y autenticada, pero el endpoint administrativo exige ADMIN. El fallo es de autorización, no de autenticación.

Resultado esperado global

- Dependencia spring-security-test disponible desde 6.4 y reutilizada sistemáticamente en 6.9.
- Tests de controladores, endpoints públicos/restringidos, login, refresh, logout y filtro JWT.
- Uso de @WithMockUser, @WithUserDetails y request post-processors donde corresponde.
- Cobertura explícita de 2xx, 401 y 403 y de errores de token.

Resumen técnico del ejercicio

- @WithMockUser y @WithUserDetails.
- SecurityMockMvcRequestPostProcessors con MockMvc.
- Tests positivos y negativos de autenticación/autorización.
- Tests de access/refresh/logout y filtro JWT.

Con este punto termina M6. El proyecto M6/6.9/proyecto será el punto de partida de M7.